CJR Core: Syntax, Semantics, and Separation Logic

Dan R. Ghica

October 29, 2026

Abstract

A tutorial on the CJR operational core, generic source layer, Iris contracts, and small extracted examples.

Contents

1 What this development is **4**

2 Tool structure and workflow **4**

3 Syntax **7**
- 3.1 Concrete syntax . . . 7
- 3.2 From Cangjie to the Coq core . . . 8
- 3.3 Locations, identifiers, and types . . . 9
- 3.4 Values and expressions . . . 10
- 3.5 Generic declarations and source expressions . . . 12
- 3.6 Two kinds of binding . . . 13
- 3.7 Generic instantiation, substitution, and erasure . . . 14

4 Machine state **15**
- 4.1 Three regions, one counter . . . 15
- 4.2 Well-formed states . . . 15

5 Small-step semantics **16**
- 5.1 Head reduction . . . 17
- 5.2 Evaluation contexts . . . 18
- 5.3 Determinism . . . 18

6 Separation logic **19**
- 6.1 What is owned . . . 19
- 6.2 Weakest precondition, in this instantiation . . . 19
- 6.3 Weakest preconditions for generic source . . . 20
- 6.4 Structural rules . . . 23
- 6.5 Pure rules . . . 24
- 6.6 Stack rules . . . 26
- 6.7 Raw-heap rules . . . 26
- 6.8 Object rules . . . 27
- 6.9 Exclusivity . . . 28

7 Main theorems **28**
- 7.1 Head reduction is deterministic . . . 28
- 7.2 Well-formedness is an invariant of the verified steps . . . 28
- 7.3 Adequacy . . . 29
- 7.4 Generic adequacy and conservative extension . . . 29

8 The examples **29**
- 8.1 Arithmetic: `arith.v` . . . 30
- 8.2 Stack structs: `struct_upd.v` . . . 30
- 8.3 One raw cell: `cell.v` . . . 31
- 8.4 Swap and aliasing: `swap.v` . . . 32
- 8.5 Indexed blocks: `sum_block.v` . . . 33
- 8.6 Final classes: `class_upd.v` . . . 34
- 8.7 Framing across a call: `frame_call.v` . . . 34

8.8 A hand-written vtable: `vtable.v` . . . 35

9 Parametric polymorphism and conservative erasure **37**
9.1 From a declaration to the program being proved . . . 37
9.2 Identity: one proof for every representation . . . 37
9.3 Pairs: independent parameters and projection resources . . . 38
9.4 Boxes: allocation and reading . . . 39
9.5 A type-changing method and its callback contract . . . 40
9.6 Concrete and nested instances . . . 41
9.7 Generated source and extraction checks . . . 41
9.8 Validation and the scope of the guarantees . . . 43

10 Limitations and target-code risks **44**

11 Migration and current runtime evidence **44**
11.1 Measurement protocol . . . 44
11.2 benchmark/results . . . 44

References **45**

1 What this development is

We take as our starting point a deliberately small core. CJR has a sequential untyped operational core and code pointers, so that a vtable can be written by hand. An explicit rank-one generic source layer erases to that same core and extracts to native Cangjie generics. Inheritance, interface bounds, higher-rank polymorphism, `enum`/`match`, and concurrency are outside this fragment. Features of a larger source language that fall outside this cut are treated as sugar that must lower into the core at the same time and space cost. What follows defines both the core and the explicit generic layer. The function `erase` defines the latter's translation to the core. There is no general Cangjie parser or lowering pass, garbage collector, or byte-level layout model.

Safety, in this development, is not a type system. The legacy grammar `ty` documents core representations. The separate grammar `gty` records structured source types, including type variables and named applications such as `Box<Pair<Int64, Bool>>`. The generic checker validates names, scopes, and arities; it is not a complete typing judgment. The operational semantics remains untyped. A program is safe when a weakest-precondition proof exists; adequacy then says that, from the empty machine, execution does not get stuck. Generic specifications quantify over Iris predicates describing element representations. They prove uniform behavior of the supplied implementations; they do not establish a general relational-parametricity theorem for all well-typed Cangjie programs.

The Coq sources live under `theories/`. The language is an Iris evaluation-context language [6, 5], so the frame rule and the bind rule are Iris's, rather than those of a separate Hoare calculus. The overall shape follows HeapLang, the example language shipped with the Iris Coq development [9].

2 Tool structure and workflow

The intended workflow starts with two related inputs. The first is an informal description of a data structure: its abstract contents, a concrete representation, and the algorithms for its operations. The second is a set of properties that should validate the implementation. These include the main representation-correctness claims (for example, that an implementation of list insertion produces the abstract list with the new element) and equational client properties (for example, that adding an element and then removing it restores the original list). Such laws are separate obligations; they do not follow merely from writing down the operations.

An engineer or an LLM can help translate the informal material into formal definitions and theorem statements. The formal development gives a mathematical model of the abstract structure, a representation predicate connecting that model to the CJR heap, executable CJR expression trees for the operations, and specifications stated as weakest preconditions. The translation is a proposal that must be reviewed: informal text does not become a theorem until its meaning has been made explicit in Rocq/Coq. For generic programs, the executable definitions are `gexpr` trees with explicit declaration references and type arguments. Their specifications use the same Iris logic after erasure to `expr`; a declaration such as `Box<T>.map<U>` is proved once against arbitrary input and output representation predicates.

The proofs use Iris's separation logic over the small-step semantics of CJR. The Iris weakest-precondition rules describe how each operation transforms owned heap resources and establishes its postcondition. The proved implementation-correctness theorems connect the concrete representation to the abstract model; algebraic laws can then be proved by composing those verified operations. Rocq/Coq's kernel checks the resulting proof terms. In this repository the development is built with Coq 8.20.1 and Iris 4.4.0. The language is called Gallina (two l's); Rocq is the prover, and Iris is the program logic and library used for the heap reasoning.

Once the CJR definitions are available, `tools/extraction/rocq/emit_cj.v` uses a custom Gallina printer to turn the selected CJR syntax trees into Cangjie source files under `example/*/extracted/`. This is not Rocq's standard extraction mechanism, and it does not extract proof terms: it serializes the program definitions. A disciplined build runs `make verify` to check proofs and assumptions, then `make cj` to emit the programs. Passing the proof check establishes the stated CJR properties; by itself it does not prove that the printer's output preserves the CJR semantics. The original printing interface remains available for existing programs. The generic path in `tools/extraction/rocq/generic_print.v` prints the structured source directly, retaining type parameters and named layouts. It does not try to reconstruct generic types from erased terms.

Finally, Cangjie unit tests compile and execute representative cases of the generated code. They are valuable sanity checks for the printer, compiler, and runtime together, but they cover finitely many inputs and do not replace the proofs. The current project contains this formal semantics, Iris proofs, custom printer, and runtime tests. It does not yet implement an automatic LLM front end that accepts arbitrary informal specifications. The distinction between the proved CJR program and the tested emitted Cangjie is important; Appendix 10 records concrete risks at that boundary.

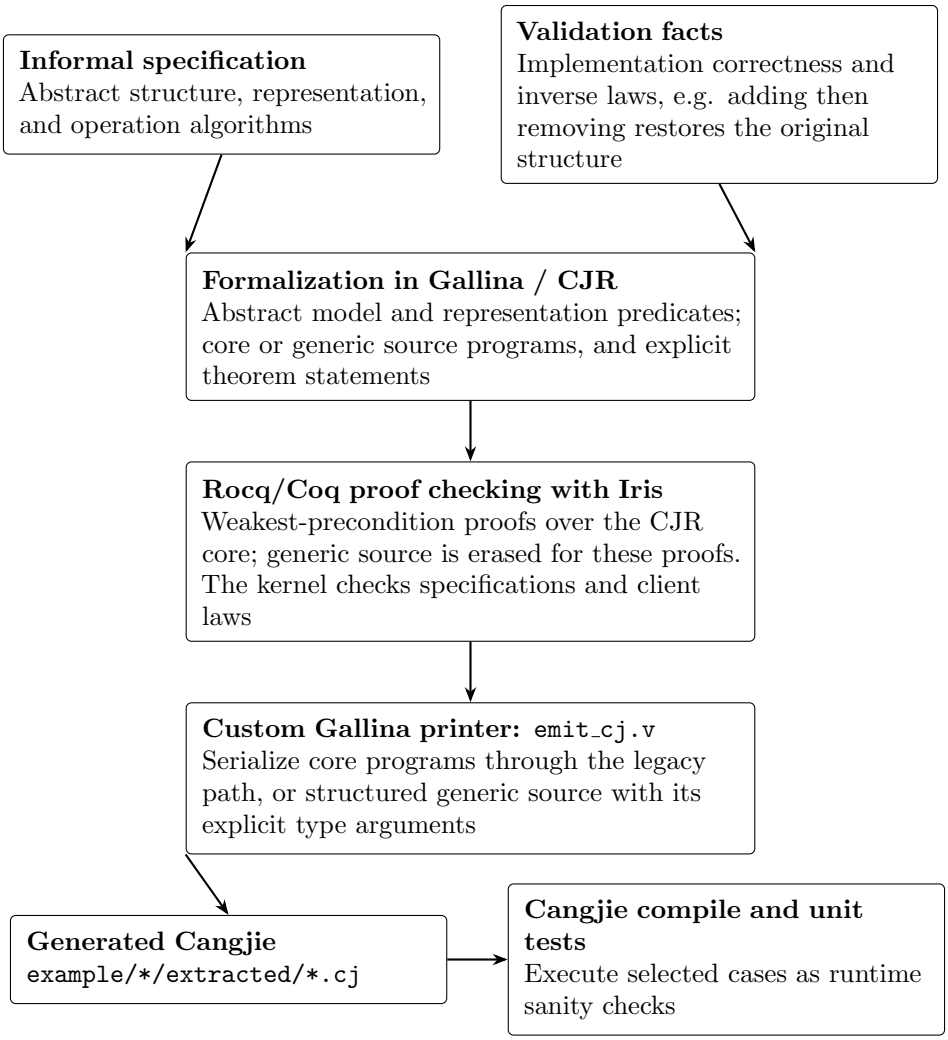


Figure 1: The intended specification-to-test workflow and the components implemented in this repository. Generic source supplies both the erased program being proved and the typed source being printed. The LLM-assisted formalization is a human review point; proof checking validates CJR theorems, while unit tests sample the separately printed Cangjie program.

3 Syntax

3.1 Concrete syntax

The programs in this note are written in Cangjie, as specified by the language documentation [1]. CJR is precisely the fragment of that language that the mechanization covers. The core in `theories/lang.v` is the lowering target, not a second surface syntax. A multi-argument `func` lowers to a single argument—a struct of the arguments—because core application is unary. A class that is not marked `open` cannot be inherited; that is the final class of the core. `CPointer` reads, writes, and offsets are the `unsafe` methods `read`, `write`, and `+` from the standard library, and allocation is the `foreign` function `malloc` from the Cangjie–C interoperability chapter.

```
// Core types: Int64, Float64, Bool, Unit, CPointer<Int64>
// Generic types: T, Pair<A, B>, Box<T>, (T) -> U
// Pair and Box declarations follow this overview.

func genericId<T>(x: T): T { x }
let answer: Int64 = genericId<Int64>(42)
let flag: Bool = genericId<Bool>(true)

func add(a: Int64, b: Int64): Int64 {
    a + b
}

let frozen: Int64 = 1
var n: Int64 = 1
while (1 <= n) {
    n = n - 1
}

if (n <= 0) {
    0
} else {
    n
}

struct Cell {
    public var x: Int64
    public init(x: Int64) { this.x = x }
}

class Counter {          // not open, so it has no subclasses
    public var value: Int64
    public init(value: Int64) { this.value = value }
}

foreign func malloc(size: UIntNative): CPointer<Unit>
foreign func free(ptr: CPointer<Unit>): Unit
```

`let` is immutable, and in the core it is substitution. `var` is mutable, and it is a stack slot. The last expression of a block is its value, and `()` is unit. Generic calls supply explicit type arguments. Below, `Pair<A, B>` has two parameters, and `Box<T>.map<U>` adds a method parameter. The callback type `(T) -> U` is monomorphic at each call.

```
struct Pair<A, B> {
    public let first: A
    public let second: B
    public init(first: A, second: B) {
        this.first = first
        this.second = second
    }
}
class Box<T> {
    public let value: T
    public init(value: T) { this.value = value }
    public func get(): T { value }
    public func map<U>(f: (T) -> U): Box<U> {
        Box<U>(f(value))
    }
}
let pair = Pair<Int64, Bool>(7, true)
let boxed = Box<Pair<Int64, Bool>>(pair)
let truth: Box<Bool> = Box<Int64>(5).map<Bool>(
    { x: Int64 => 0 <= x })
```

These are readable surface forms of the checked generic examples. Section 9 gives their source definitions, proofs, and the generated code. The generic layer currently supports immutable bindings, typed callbacks, conditionals, calls, struct construction/projection, and class allocation/read, and initialized managed buffers with indexed reads and writes (the polymorphic-array chapter of the CJR Collections tutorial). The mutable and raw-memory forms above remain available through the original language and printer.

3.2 From Cangjie to the Coq core

We summarise the original core correspondence in the table below. Each row pairs one Cangjie form with the constructor in `theories/lang.v` to which it lowers. Field names become positions: the first field of a `struct` or `class` is index 0. A call `f(a, b)` is `App` of `f` to a `StructV` of the arguments, because `App` takes one argument. Forms that mention a string name of a `var` are rewritten by `subst_var` into the administrative stack operations listed in the last rows. Those operations have no spelling in Cangjie; they appear only after a `var` has been given a stack slot.

Cangjie	Coq constructor
`Int64`, `Float64`, `Bool`, `Unit`	`TyInt`, `TyFloat`, `TyBool`, `TyUnit`
`struct S { ... }`	`TyStruct` of the field types
`CPointer<T>`	`TyPtr`
`class C { ... }` with no `open`	`TyObj` of the field count
`(T) -> U`	`TyFun`
`1`, `1.0`, `true`, `()`	`LitInt`, `LitFloat`, `LitBool`, `LitUnit`
a `CPointer` value, including null from `CPointer<T>()`	`LitPtr`, `LitNull`
a class instance	`LitObj`
`func f(x) { e }`	`Rec` (some name, some parameter, body); a finished closure is `RecV`
a struct value such as `Cell(0)`	`StructV`
a name	`Var`
`let x = e1` followed by `e2`	`Let`
`var x = e1` followed by `e2`	`VarBind`
`x = e` for a `var x`	`Assign`
`if (e0) { e1 } else { e2 }`	`If`
`while (e1) { e2 }`	`While`
two expressions in sequence, or a block	`Seq`
`+`, `-`, `%`, `<=`, `==` on `Int64`, `&&`	`PlusOp`, `MinusOp`, `MulOp`, `RemOp`, `LeOp`, `EqOp`, `AndOp`
`^`, `&` on `Int64`	`BitXorOp`, `BitAndOp`
`f(a)`	`App`
`Cell(e)`	`Struct`, evaluated left to right
`p.x` on a struct	`StructLoad` at the index of `x`
`p.x = e` on a `var` struct	`StructStore` of that variable and index
`Counter(e)`	`New`
`o.value`	`FieldLoad`
`o.value = e`	`FieldStore`
`malloc` of one word, then `CPointer<Int64>(...)`	`Alloc` of `1`, result `LitPtr`
`free(p)`	`Free`
`unsafe { p.read() }`	`Load`
`unsafe { p.write(v) }`	`Store`
`unsafe { p + i }`	`Offset`
no Cangjie spelling; introduced for a `var` slot	`Pop`, `StackLoad`, `StackAssign`, `StackFieldStore`, and the literal `LitStack`

Generic programs first pass through the explicit source layer. A call `genericId<Int64>(42)` is a `GECall` containing a declaration name, the type argument `GTInt`, and an argument expression. `Pair<Int64, Bool>(a, b)` and `Box<Int64>(a)` use `GEConstruct` with the named owner types; they erase to `Struct` and `New`, respectively. A field access uses `GEField` with its receiver type and declared index. A method call uses `GEMethod` and passes the receiver as the first internal argument. Section 3.7 gives the translation and its laws.

3.3 Locations, identifiers, and types

A location is an integer. We wrap stack slots, raw words, and objects in distinct constructors so that the four ghost maps of the program logic cannot be confused.

```
Definition loc : Type := Z.
Inductive stack_id := StackId (z : Z).
Inductive raw_id := RawId (z : Z).
Inductive obj_id := ObjId (z : Z).
```

```
Inductive ty :=
  | TyInt | TyFloat | TyBool | TyUnit
  | TyStruct (ts : list ty)
  | TyPtr (t : ty)
  | TyObj (nfields : nat)
  | TyFun (t1 t2 : ty).
```

Informally, locations are $\mathbb{Z}$, and the three identifier sorts are the tagged copies $\mathsf{StackId}(\ell)$, $\mathsf{RawId}(\ell)$, and $\mathsf{ObjId}(\ell)$. The legacy core type grammar is

$$\tau ::= \mathsf{Int64} \mid \mathsf{Float64} \mid \mathsf{Bool} \mid \mathsf{Unit} \mid \mathsf{struct}\,\overline{\tau} \mid \mathsf{CPointer}\,\tau \mid \mathsf{class}_n \mid \tau \to \tau.$$

Here class_n means a final class with n fields. Nothing in the semantics consults τ. The type is documentation of the fragment; the theorems that follow do not assume it.

The generic source has a separate, structured grammar in `theories/generics.v`:

```
Inductive gty :=
| GTInt | GTBool | GTUnit
| GTVar (name : string)
| GTNamed (name : string) (args : list gty)
| GTArrow (arg ret : gty)
| GTBuffer (element : gty).
```

Writing α for a scoped type variable and N for a declared struct or class name, its mathematical grammar is

$$\begin{aligned} t ::= {} & \mathsf{Int64} \mid \mathsf{Bool} \mid \mathsf{Unit} \mid \alpha \\ & \mid N\langle t_1, \ldots, t_n \rangle \mid t \to t \mid \mathsf{Buffer}\langle t \rangle. \end{aligned}$$

Thus `Box<Pair<Int64, Bool>>` is a nested named application, and `(T) -> U` is an arrow whose endpoints can be in-scope variables. There is no universal quantifier inside `gty`: polymorphism is rank one and belongs to declarations. There are no generic pointer or float constructors in this source grammar; their legacy core forms are unchanged. The buffer constructor supports managed indexed storage for the separate polymorphic-array example in the polymorphic-array chapter of the CJR Collections tutorial.

`gty_wf ds scope t` checks that every variable occurs in `scope`, every named type has a layout in `ds`, and its number of arguments matches that layout's parameter list. It recursively checks nested arguments, buffer elements, and both sides of an arrow. This is a well-formedness check on types, not a proof that a term has that type. Type substitution `gty_subst` replaces variables simultaneously: $\{T \mapsto U, U \mapsto \mathsf{Int64}\}(T) = U$, not $\mathsf{Int64}$. Nested applications are traversed recursively.

3.4 Values and expressions

A binder is an optional name: None is an ignored parameter. Literals cover integers, a float payload (for which no float operations are defined), booleans, unit, raw pointers, null, object references, and stack addresses. Values are literals, recursive closures, and unboxed struct tuples. Expressions comprise the core forms, together with administrative forms (`Pop`, `StackLoad`, `StackAssign`, `StackFieldStore`) that are introduced when a mutable variable is allocated.

```
Definition binder := option string.

Inductive base_lit :=
```

```
  | LitInt (n : Z) | LitFloat (n : Z) | LitBool (b : bool)
  | LitUnit | LitPtr (l : loc) | LitNull
  | LitObj (o : loc) | LitStack (l : loc).

Inductive val :=
  | LitV (l : base_lit)
  | RecV (f x : binder) (e : expr)
  | StructV (vs : list val)
with expr :=
  | Val (v : val) | Var (x : string)
  | Rec (f x : binder) (e : expr)
  | App (e1 e2 : expr)
  | Let (x : binder) (e1 e2 : expr)
  | VarBind (x : string) (e1 e2 : expr)
  | Assign (x : string) (e : expr)
  | Pop (l : stack_id) (e : expr)
  | StackLoad (e : expr)
  | StackAssign (e1 e2 : expr)
  | StackFieldStore (e1 : expr) (i : nat) (e2 : expr)
  | If (e0 e1 e2 : expr)
  | While (e1 e2 : expr)
  | Seq (e1 e2 : expr)
  | BinOp (op : bin_op) (e1 e2 : expr)
  | Alloc (e : expr) | Free (e : expr)
  | Load (e : expr) | Store (e1 e2 : expr)
  | Offset (e1 e2 : expr)
  | Struct (vs : list val) (es : list expr)
  | StructLoad (e : expr) (i : nat)
  | StructStore (x : string) (i : nat) (e : expr)
  | New (vs : list val) (es : list expr)
  | FieldLoad (e : expr) (i : nat)
  | FieldStore (e1 : expr) (i : nat) (e2 : expr).
```

In mathematics we write values

$$v ::= n \mid f \mid b \mid () \mid \mathsf{ptr}(\ell) \mid \mathsf{null} \mid \mathsf{obj}(\ell) \mid \mathsf{stk}(\ell) \mid \mathsf{rec}\, f\, x.\, e \mid \langle v_0, \ldots, v_{k-1} \rangle$$

and we use the expression forms by the same names as the constructors.

```
Inductive bin_op :=
  PlusOp | MinusOp | MulOp | RemOp | LeOp | EqOp | AndOp
  | BitXorOp | BitAndOp.
```

BinOp ranges over $+$, $-$, $*$, signed remainder, $\leq$, $=$, boolean conjunction, integer bitwise XOR, and integer bitwise AND. Integer operations act on LitInt; AndOp acts on booleans. The two bitwise evaluator cases are:

```
| BitXorOp, LitV (LitInt n1), LitV (LitInt n2) =>
    Some (LitV (LitInt (Z.lxor n1 n2)))
| BitAndOp, LitV (LitInt n1), LitV (LitInt n2) =>
    Some (LitV (LitInt (Z.land n1 n2)))
```

Thus the semantics uses Rocq's unbounded signed integers, including their bitwise operations on negative values; it does not impose a 64-bit truncation. For byte-valued operands, XOR stays

within $0 \ldots 255$, and AND with 255 selects the low eight bits. For example, 128 XOR $27 = 155$ and 511 AND $255 = 255$. The expression notations are `e1 .^ e2` and `e1 .& e2`; the printer emits Cangjie `^` and `&`. Boolean conjunction keeps its separate `&&` spelling. Bitwise OR, complement, and shifts are outside the current syntax. Anything else makes bin_op_eval return None, and the machine gets stuck.

Structs and objects are built by a list zipper. $\mathsf{Struct}\,\overline{v}\,\overline{e}$ has already evaluated the values $\overline{v}$ and still has the expressions $\overline{e}$ to evaluate, left to right. New is the same zipper for a final class. When the expression list is empty, the zipper is done.

`Val` and `to_val` identify which expressions are finished:

```
Definition of_val (v : val) : expr := Val v.
Definition to_val (e : expr) : option val :=
  match e with Val v => Some v | _ => None end.
```

So the values of the language, as Iris sees them, are exactly the expressions $\mathsf{Val}\,v$.

3.5 Generic declarations and source expressions

A `glayout` names a struct or class, gives its type-parameter list, and fixes the ordered field names and types. A `gfunction` records a name, an optional owning layout for methods, a static-method flag, its own type parameters, ordinary parameters, result type, and `gexpr` body. A `gprogram` combines layouts, functions, and a main expression. The example definitions include

```
Definition pair_layout :=
  GLayout "Pair" GStruct ["A"; "B"]
    [("first", GTVar "A"); ("second", GTVar "B")].
Definition box_layout :=
  GLayout "Box" GClass ["T"] [("value", GTVar "T")].
```

The scope of a method body is its owner's parameters followed by its own. For `Box<T>.map<U>`, this is $[T, U]$; rebinding `T` as a method parameter is rejected. The first ordinary parameter of an instance method is an explicit receiver of its owning type, such as `self: Box<T>`. The printer renders this parameter as `this`. A static method has no receiver; `GStaticFunction` declares it and `GEStaticCall` calls it through an explicit owner type, as in `PolyArray<Bool>.make(3, false)` (the polymorphic-array chapter of the CJR Collections tutorial).

The source constructors preserve the information that erasure discards:

```
GECall name type_arguments arguments
GEMethod receiver_type receiver method type_arguments arguments
GEStaticCall owner_type method type_arguments arguments
GEConstruct named_type arguments
GEField receiver_type receiver field_index
GELam parameter argument_type result_type body
GEBufferMake element_type size seed
GEBufferGet element_type buffer index
GEBufferSet element_type buffer index value
```

These schematic applications accompany `GEVar`, `GEVal`, `GEApp`, `GELet`, `GEIf`, and `GEBinOp`. `GEField` carries an index into the explicitly named layout; its field spelling is recovered from that declaration, not from its size. For example, the body of `Box<T>.map<U>` is

```
GEConstruct (GTNamed "Box" [GTVar "U"])
```

```
  [GEApp (GEVar "f")
    (GEField (GTNamed "Box" [GTVar "T"]) (GEVar "self") 0)]
```

A callback has a fixed argument and result type at each call site. There is no argument whose type is itself universally quantified. To pass a generic function as a callback, write a typed lambda that calls it with explicit type arguments. The generic printer rejects bare `GERef` function values. It also rejects `GECore`, which is an exact embedding of old core terms for proofs, rather than a printable generic source form. This distinction preserves all legacy specifications without claiming the new printer handles every old expression.

3.6 Two kinds of binding

We begin from a distinction that is already present in the surface language and then explain how the core encodes it. Immutable `let` and function parameters are substitution; mutable `var` is not. A `var` is given a fresh stack slot, and every use or assignment of that name is rewritten into an operation on the slot. A bare struct value is still copied: mutation in place happens only through a `var` slot or through a stored field.

Those surface facts come from the Cangjie 1.0 user manual [1]. The stack-slot encoding below is this note's model of them, not a spelling in the language.

- **Immutable vs mutable bindings.** The chapter *Program Structure* (manual) introduces the mutability modifiers `let` and `var`: a `let` variable is assigned once (initialization), whereas a `var` variable may be reassigned. The chapter *Function Definition* (manual) states that function parameters are immutable and cannot be assigned to in the function body.

- **Struct copying.** The same *Program Structure* chapter distinguishes value types (including `struct`) from reference types (including `class`): assigning a value-type variable copies the instance; later mutation of one copy does not affect the other. *Creating a struct Instance* (manual) repeats that assignment and parameter passing copy a `struct`, and that changing fields through an instance requires both the struct variable and the fields to be `var`. The related page *mut Function* (manual) adds that `mut` methods need a mutable (`var`) receiver; they cannot be called on a `let` struct.

- **Stored class fields.** The chapter *Classes* (manual) states that `class` is a reference type: assignment does not copy the object, so field updates through one variable are visible through others that share it. That is the "stored field" case in the sentence above.

What follows—fresh stack slots, `subst_var`, `StackLoad`/`StackAssign`/`StackFieldStore`, `LitStack`, and `Pop`—is the Coq core's administrative rewriting, not Cangjie surface syntax.

```
Definition subst_binder (b : binder) (v : val) (e : expr) : expr :=
  match b with None => e | Some x => subst x v e end.
```

The function $\mathsf{subst}\,x\,v\,e$ replaces free occurrences of x by the value v, and it does not descend under a binder that captures x. On a value, literals are unchanged and the body of a closure is substituted only when x is not the function name or the parameter.

The mutable variant $\mathsf{subst_var}\,x\,\ell\,e$ does a different job. A read $\mathsf{Var}\,x$ becomes $\mathsf{StackLoad}(\mathsf{stk}(\ell))$. An assignment $\mathsf{Assign}\,x\,e$ becomes $\mathsf{StackAssign}(\mathsf{stk}(\ell), e')$. A field write $\mathsf{StructStore}\,x\,i\,e$ becomes $\mathsf{StackFieldStore}(\mathsf{stk}(\ell), i, e')$. Other occurrences of x are rewritten recursively, again respecting capture. The slot address is itself a value, so later stack operations do not look up a string environment.

For a concrete example, the fragment $\mathsf{var}\ x = 1\ \mathsf{in}\ (x = x + 1)$ allocates a fresh slot ℓ, stores 1, and rewrites the body, reaching

$$\mathsf{Pop}\,(\mathsf{StackId}(\ell))\,(\mathsf{StackAssign}(\mathsf{stk}(\ell),\ \mathsf{BinOp}+(\mathsf{StackLoad}(\mathsf{stk}(\ell)),\ 1))).$$

Informally that is $\mathsf{pop}_\ell\,(\mathsf{stk}(\ell) := \mathsf{stk}(\ell) + 1)$: the read of x is a load from the slot, the assignment writes the slot, and Pop deletes it when the body finishes.

What is striking about this split is that it is the whole model of "locals do not need a points-to fact until they are mutable." An immutable binding disappears into the syntax. A mutable binding is a cell, and the cell has an owner.

3.7 Generic instantiation, substitution, and erasure

Type instantiation is separate from term substitution. `ginstantiate s e` applies `gty_subst s` to every type annotation and explicit type argument in `e`. It leaves term variables and values alone. `gsubst x v e` replaces free occurrences of the term variable `x` by the core value `v`, respecting lambda and immutable-let binders. It leaves the type annotations alone.

Let D be the layout declarations, F an environment of linked function values, and $E(e) = \mathsf{erase}\ D\ F\ e$. Representative erasure equations are

$$\begin{aligned}
E(\mathsf{GECall}\ n\ \bar{t}\ \bar{e}) &= \mathsf{App}\ (\mathsf{resolve}\ F\ n)\ (\mathsf{Struct}\ []\ \overline{E(e)}),\\
E(\mathsf{GEConstruct}\ (\mathsf{Pair}\langle a, b\rangle)\ \bar{e}) &= \mathsf{Struct}\ []\ \overline{E(e)},\\
E(\mathsf{GEConstruct}\ (\mathsf{Box}\langle t\rangle)\ [e]) &= \mathsf{New}\ []\ [E(e)],\\
E(\mathsf{GEField}\ (\mathsf{Box}\langle t\rangle)\ e\ 0) &= \mathsf{FieldLoad}\ (E(e))\ 0.
\end{aligned}$$

Here `Pair` and `Box` have the declarations above. A struct projection similarly becomes `StructLoad`. A method call resolves the key `Box.map` and packs the receiver before its ordinary arguments. A typed lambda becomes `Rec None (Some x)` with its erased body, and uses ordinary unary application. No type argument becomes a runtime value or heap field.

`link_functions` processes declarations in order. Previously declared functions resolve to core recursive values. A function's own name remains a variable in its body and is bound by its `RecV`, preserving direct recursion. The checker rejects other forward references and mutual recursion. Parameter unpacking uses internal struct projections; argument expressions evaluate from left to right before the body runs.

Two kernel-checked equalities justify the connection between source reasoning and the old substitution rules:

```
Lemma erase_instantiate ds fs e s : nominal_owners e = true ->
  erase ds fs (ginstantiate s e) = erase ds fs e.

Lemma erase_subst ds fs e x v :
  (forall n, In n (referenced_names e) ->
    subst x v (resolve fs n) = resolve fs n) ->
  erase ds fs (gsubst x v e) = subst x v (erase ds fs e).
```

The first premise requires constructor and receiver annotations to be named applications, not bare variables. Substitution can change the arguments in `Box<T>` but cannot change `Box` into a struct. Without that restriction, instantiation could change allocation into tuple construction. The second premise says that named declarations are stable under substitution for the local variable; globals

must not accidentally be treated as local occurrences. Both laws are proved by structural recursion, using the corresponding laws for lists of arguments.

The legacy embedding satisfies the unconditional equation $E(\mathsf{GECore}\ e) = e$, proved by `erase_legacy`. Consequently the extension includes every old core program exactly.

4 Machine state

4.1 Three regions, one counter

The machine is a single record. Locals and unboxed structs that are stored in `var` slots live in the stack map. `CPointer` words live in the raw map, and each allocated block records its length in a second map on the same addresses. Final class fields live in a map from $(\mathsf{ObjId}(\ell), i)$ to a value. A single counter `next` hands out fresh addresses for all three regions. Generic instances use these same representations: a `Pair<A, B>` is a two-value struct, and a `Box<T>` is an object whose field zero stores its element value. A nested box contains the inner object's reference, not a copy of the inner heap. Type arguments introduce neither runtime type tags nor a new region in this model.

```
Record state := MkState {
  cjr_stack  : gmap stack_id val;
  cjr_raw    : gmap raw_id val;
  cjr_blocks : gmap raw_id nat;
  cjr_obj    : gmap (obj_id * nat) val;
  cjr_next   : Z
}.
Definition state_init : state := inhabitant. (* all maps empty, next = 0 *)
```

Write a state as

$$\sigma = (\sigma_s,\, \sigma_r,\, \sigma_b,\, \sigma_o,\, \mathsf{next}(\sigma)).$$

The initial state σ_0 is $(\emptyset, \emptyset, \emptyset, \emptyset, 0)$.

Indexing a pointer is arithmetic on the address, one word per element:

$$\mathsf{ptr}(\ell) + i \;=\; \mathsf{ptr}(\ell + i).$$

There is no byte offset and no padding. A block of length n at base ℓ occupies the words $\ell, \ell+1, \ldots, \ell+n-1$. Blocks are freed explicitly. Class objects are not freed in this development: there is no `delete` for them, and there is no reachability argument.

Null is a pointer value, not an address in σ_r. Loading or storing through `null` has no rule, so it is stuck. An out-of-bounds or use-after-free access is stuck for the same reason: the lookup premise of load, store, or free fails.

4.2 Well-formed states

Not every 5-tuple is a state that the allocator is allowed to produce. Well-formedness says that every address already in use lies strictly below the counter, that every block's words are present, and that live blocks do not overlap.

```
Definition stack_below m n :=
  forall l, StackId l in dom m -> (l < n)%Z.
(* raw_below, block_below, obj_below are the same idea. *)
```

```
Definition block_cover blocks raw next :=
  forall l n, blocks !! RawId l = Some n ->
    (l + Z.of_nat n <= next)%Z /\
    forall i, i < n -> is_Some (raw !! RawId (l + Z.of_nat i)).

Definition blocks_disjoint blocks :=
  forall l1 n1 l2 n2,
    blocks !! RawId l1 = Some n1 ->
    blocks !! RawId l2 = Some n2 ->
    l1 <> l2 ->
    (l1 + Z.of_nat n1 <= l2 \/ l2 + Z.of_nat n2 <= l1)%Z.

Record state_wf (s : state) : Prop := {
  wf_stack : stack_below (cjr_stack s) (cjr_next s);
  wf_raw   : raw_below (cjr_raw s) (cjr_next s);
  wf_block : block_below (cjr_blocks s) (cjr_next s);
  wf_obj   : obj_below (cjr_obj s) (cjr_next s);
  wf_cover : block_cover (cjr_blocks s) (cjr_raw s) (cjr_next s);
  wf_disj  : blocks_disjoint (cjr_blocks s)
}.
```

In mathematics, $\mathsf{wf}(\sigma)$ is the conjunction of:

$\forall \ell.\, \mathsf{StackId}(\ell) \in \mathrm{dom}(\sigma_s) \Rightarrow \ell < \mathsf{next}(\sigma)$,

and the same for raw addresses, block bases, and object addresses,

$\forall \ell, n.\, \sigma_b(\mathsf{RawId}(\ell)) = \mathsf{Some}(n) \Rightarrow \ell + n \leq \mathsf{next}(\sigma) \ \wedge\ \forall i < n.\, \sigma_r(\mathsf{RawId}(\ell + i)) \neq \mathsf{None}$,

distinct live blocks have disjoint integer intervals.

The freshness lemmas are immediate from the first four clauses. If $\mathsf{wf}(\sigma)$ and $\ell = \mathsf{next}(\sigma)$, then ℓ is absent from the stack, from the raw heap, from the block map, and, at every field index, from the object heap. Allocation is therefore the insertion of a key that is not already present.

The initial state is well formed: `state_init_wf`. Each successful head step that the program logic uses is proved to preserve wf: stack allocation, deletion, and update; raw store; one-word allocation and freeing; class allocation; and field update.

One-word allocation is worth spelling out, because it is the case the logic rule talks about. From base $\ell = \mathsf{next}(\sigma)$,

$$\begin{aligned} \sigma_r' &= \{\mathsf{RawId}(\ell) \mapsto 0\} \cup \sigma_r, \\ \sigma_b' &= \sigma_b[\mathsf{RawId}(\ell) \mapsto 1], \\ \mathsf{next}' &= \ell + 1. \end{aligned}$$

The new word is in range, the new block is covered, and disjointness holds because the new interval $[\ell, \ell + 1)$ starts at the old counter, past every previous address. Freeing that block deletes the length and the word. Other blocks keep their cells precisely because their intervals were disjoint from $[\ell, \ell + 1)$.

Class allocation at $o = \mathsf{next}(\sigma)$ inserts the fields $(\mathsf{ObjId}(o), i) \mapsto v_i$ for a list $v_0, \ldots, v_{k-1}$ and advances the counter by one. Those keys are fresh because every old object address is below o.

5 Small-step semantics

The relation in this section executes core expressions. Execution of a generic program p means execution of `erase_program p` with these same rules. There is no additional type-instantiation

reduction: the erased code is already independent of type arguments. This definition does not assert that the Cangjie compiler uses the same runtime representation.

5.1 Head reduction

A head step is a relation

$$(e, \sigma) \longrightarrow (e', \sigma')$$

with no observations and no forked threads. Both are hard-wired: `observation` is the empty type, and every rule produces an empty list of threads. The language is deterministic on the nose.

Pure computation. These rules neither read nor write the heaps.

$$\begin{aligned}
\mathsf{rec}\, f\, x.\, e &\longrightarrow \mathsf{rec}\, f\, x.\, e && \text{(as a value)} \\
(\mathsf{rec}\, f\, x.\, e)\, v &\longrightarrow e[x \mapsto v][f \mapsto \mathsf{rec}\, f\, x.\, e] && \\
\mathsf{let}\, x = v \mathsf{in} e &\longrightarrow e[x \mapsto v] && \\
\mathsf{if\ true\ then}\ e_1\ \mathsf{else}\ e_2 &\longrightarrow e_1 && \\
\mathsf{if\ false\ then}\ e_1\ \mathsf{else}\ e_2 &\longrightarrow e_2 && \\
\mathsf{while}\ e_1\ \mathsf{do}\ e_2 &\longrightarrow \mathsf{if}\ e_1\ \mathsf{then}\ (e_2; \mathsf{while}\ e_1\ \mathsf{do}\ e_2)\ \mathsf{else}\ () && \\
v;\, e &\longrightarrow e && \\
v_1 \oplus v_2 &\longrightarrow v && \text{when } [\![\oplus]\!](v_1, v_2) = \mathsf{Some}(v) \\
\mathsf{ptr}(\ell) + i &\longrightarrow \mathsf{ptr}(\ell + i) && \\
\langle \overline{v},\, v :: \overline{e} \rangle &\longrightarrow \langle \overline{v} {+\!+} [v],\, \overline{e} \rangle && \\
\langle \overline{v} \rangle &\longrightarrow \langle v_0, \ldots \rangle && \text{(the struct value)} \\
\langle \overline{v} \rangle.i &\longrightarrow v && \text{when } v \text{ is the } i\text{th component.}
\end{aligned}$$

The `while` rule duplicates the loop. It is a head step, not an evaluation context, so the copies are syntax rather than a shared mutable closure. The condition and the body are evaluated later, by ordinary contexts.

Stack. Let $\ell = \mathsf{next}(\sigma)$.

$$\begin{aligned}
\mathsf{var}\ x = v\ \mathsf{in}\ e &\longrightarrow \mathsf{pop}_\ell\, (e\{x \mapsto \ell\}) && \text{and } \sigma_s(\mathsf{StackId}(\ell)) := v,\ \mathsf{next} := \ell + 1 \\
\mathsf{pop}_\ell\, v &\longrightarrow v && \text{and delete the slot} \\
\mathsf{stk}(\ell) &\longrightarrow v && \text{if the slot holds } v \\
\mathsf{stk}(\ell) := v &\longrightarrow () && \text{if the slot exists; it then holds } v \\
\mathsf{stk}(\ell).i := v &\longrightarrow () && \text{if the slot holds a struct and index } i \text{ is in range.}
\end{aligned}$$

The last rule replaces component i with $\mathsf{set_nth}$ and writes the struct back into the same slot. No raw or object address is allocated. The most important consequence is that a stack update can be framed against a raw points-to fact: the two maps are different components of σ.

Raw heap. For $n > 0$, allocation of n words at $\ell = \mathsf{next}(\sigma)$ returns $\mathsf{ptr}(\ell)$, records length $Z.\mathsf{to_nat}(n)$, zeroes the words, and sets the counter to $\ell + n$. The proved program-logic rule

specialises this to $n = 1$.

$\mathsf{free}(\mathsf{ptr}(\ell)) \longrightarrow ()$	if a block length is recorded at ℓ; that block's words are deleted
$!\,\mathsf{ptr}(\ell) \longrightarrow v$	if the word holds v
$\mathsf{ptr}(\ell) := v \longrightarrow ()$	if the word exists; it then holds v.

Free deletes whatever length is stored. The one-word reasoning rule asks separately for the length fact 1, so the client of that rule cannot free a longer block by mistake.

Objects. The object zipper steps like the struct zipper. When it finishes, at $o = \mathsf{next}(\sigma)$,

$$\mathsf{new}(v_0, \ldots, v_{k-1}) \longrightarrow \mathsf{obj}(o)$$

and the fields $(o, i) \mapsto v_i$ are added. Field load and field store are lookup and update at $(\mathsf{ObjId}(o), i)$. Store requires the field to exist already. A missing field, like a missing word, is stuck.

5.2 Evaluation contexts

Head steps fire only at the root. The context items say where a subexpression may be reduced. Each item has one hole, and the hole is the expression currently being evaluated. Arguments are evaluated left to right. `if` evaluates only the condition. `while` does not evaluate under itself: the unfolding rule is the head step above.

Representative items, written as contexts:

$$E[\mathsf{let}\, x = [\,]\ \mathsf{in}\ e],\ E[\mathsf{var}\, x = [\,]\ \mathsf{in}\ e],\ E[\mathsf{pop}_\ell\,[\,]],$$
$$E[[\,]\,e],\ E[v\,[\,]],\ E[\mathsf{if}\,[\,]\ \mathsf{then}\ e_1\ \mathsf{else}\ e_2],\ E[[\,];e],$$
$$E[[\,] \oplus e],\ E[v \oplus [\,]],\ E[!\,[\,]],\ E[\mathsf{free}\,[\,]],\ E[[\,] := e],\ E[v := [\,]].$$

Struct and `new` items evaluate the next expression in the zipper. The Coq constructor list is `ectx_item`, and `fill_item` implements the grammar.

Iris then builds the language in the standard way.

```
Lemma cjr_ectxi_mixin : EctxiLanguageMixin of_val to_val fill_item base_step.
Canonical Structure cjr_ectxi_lang := EctxiLanguage cjr_ectxi_mixin.
Canonical Structure cjr_ectx_lang := EctxLanguageOfEctxi cjr_ectxi_lang.
Canonical Structure cjr_lang := LanguageOfEctx cjr_ectx_lang.
```

A configuration steps by finding a context K and a head redex:

$$K[e] \longrightarrow K[e'] \quad \text{when} \quad (e, \sigma) \longrightarrow (e', \sigma').$$

Filling a non-value never produces a value, a value never head-reduces, and a head reduction of a filled term implies that the hole was already a value. Those are the mixin obligations; they are what let Iris treat K as a program context in the bind rule.

5.3 Determinism

```
Lemma base_step_det e s k e' s' t k2 e2 s2 t2 :
  base_step e s k e' s' t ->
  base_step e s k2 e2 s2 t2 ->
  k = k2 /\ e' = e2 /\ s' = s2 /\ t = t2.
```

Theorem 1 (Determinism of head reduction)**.** *If* $(e, \sigma) \longrightarrow (e_1, \sigma_1)$ *and* $(e, \sigma) \longrightarrow (e_2, \sigma_2)$*, then* $e_1 = e_2$ *and* $\sigma_1 = \sigma_2$*. Observations and forked threads, which are always empty, agree as well.*

The proof is inversion of both derivations. Where a rule reads a map or calls $[\![\oplus]\!]$, the two lookups are the same, so the two results are the same. There is no scheduler and no choice of fresh address other than $\mathsf{next}(\sigma)$.

6 Separation logic

6.1 What is owned

The program logic does not talk about the whole machine. It talks about the cells a piece of code is allowed to use. Ownership is a ghost map—one authoritative map per region—with a points-to fact for each cell. Full ownership is the fraction 1.

```
Definition stack_mapsto (l : stack_id) (v : val) : iProp S :=
  ghost_map_elem stack_name l (DfracOwn 1) v.
Definition raw_mapsto (l : raw_id) (v : val) : iProp S :=
  ghost_map_elem raw_name l (DfracOwn 1) v.
Definition block_mapsto (l : raw_id) (n : nat) : iProp S :=
  ghost_map_elem block_name l (DfracOwn 1) n.
Definition obj_mapsto (o : obj_id) (i : nat) (v : val) : iProp S :=
  ghost_map_elem obj_name (o, i) (DfracOwn 1) v.
```

The notations in the sources are $\ell \mapsto_s v$, $\ell \mapsto_r v$, $\ell \mapsto_b n$, and $o \mapsto_o [i]v$. We use four maps, rather than a single heap, because a stack slot, a raw word, a block length, and a class field are different kinds of cell. A stack proof cannot accidentally be applied to a pointer.

The state interpretation ties the ghost maps to the concrete machine and remembers well-formedness:

```
Definition cjr_state_interp (s : state) : iProp S :=
  ghost_map_auth stack_name 1 (cjr_stack s) *
  ghost_map_auth raw_name 1 (cjr_raw s) *
  ghost_map_auth block_name 1 (cjr_blocks s) *
  ghost_map_auth obj_name 1 (cjr_obj s) *
  pure (state_wf s).
```

At every step, Iris owns $\mathsf{stateInterp}(\sigma)$. A points-to fact is a fragment of one of those authoritative maps. Updating a cell is updating both the fragment and the matching entry of σ, and the wf conjunct is re-established by the lemma for that step.

6.2 Weakest precondition, in this instantiation

The Iris instance is the standard one for a deterministic language with no thread pool of interest:

$$\begin{aligned} \mathsf{stateInterp}(\sigma) &= \mathsf{cjr_state_interp}(\sigma), \\ \mathsf{forkPost}(_) &= \top, \\ \mathsf{numLatersPerStep}(_) &= 0. \end{aligned}$$

The weakest precondition $\mathsf{wp}\, e\, \{\Phi\}$ is Iris's. For this tutorial the reading is:

Definition 2 (Specification)**.** $\mathsf{wp}\, e\, \{\Phi\}$ *means: for every well-formed extension of the current ghost ownership, executing* e *neither gets stuck nor forks, and if it terminates with a value* v *then* $\Phi(v)$ *holds. Partial correctness is enough: a diverging loop satisfies a weakest precondition whose postcondition is never reached.*

The later $\triangleright$ and the credit $£1$ appearing in the rules are Iris's accounting for steps, in the mode `HasLc`. When a rule says

$$\ell \mapsto_r v \mathrel{-\!\!*} \triangleright(£1 \mathrel{-\!\!*} \ell \mapsto_r v \mathrel{-\!\!*} \Phi(v)) \vdash \mathsf{wp}\,(!\,\ell)\,\{\Phi\},$$

the intended content is appealingly simple: ownership of the cell is enough to load it, and one gets the same cell back, together with the value that was there. The later and the credit are discharged by the step. In the example proofs they are thrown away with an introduction `iIntros "> _"`. They do not change the memory reasoning.

6.3 Weakest preconditions for generic source

Consider the function `genericId<T>(x: T): T { x }`. We want one proof explaining why it returns its input, whether that input is an integer, a Boolean, or a reference to an object whose fields we own. The generic logic lets us state that proof against the generic source program while using the existing rules for the code it executes.

Reading the definition. The definition in `theories/generic_laws.v` is

```
Definition generic_wp ds fs e (Phi : val -> iProp Σ) : iProp Σ :=
  WP erase ds fs e {{ Phi }}.
```

There are four inputs. The layout environment `ds` tells us, for example, that `Pair` is a struct with two fields and `Box` is a class with one field. The linked function environment `fs` supplies the executable values of named functions and methods. The expression `e` is generic source syntax. Finally, `Phi` describes the result we want, together with any resources we want back. The type `iProp` means an Iris assertion: it can describe both a mathematical fact and ownership of memory. The parameter Σ fixes the available ghost-state resources.

We write the definition mathematically as

$$\mathsf{gwp}_{D,F}\, e\, \{\Phi\} \;\triangleq\; \mathsf{wp}\,(\mathsf{erase}\ D\ F\ e)\,\{\Phi\}.$$

Read it as: execute the erased program safely, and if it returns v, establish $\Phi(v)$. This is the same partial-correctness reading as in Section 6.2; it does not promise termination. In a specification $P \vdash \mathsf{gwp}\, e\, \{\Phi\}$, the assertion P supplies enough initial facts and resources to justify that execution. A proof of such an entailment need not calculate the smallest possible precondition.

The result v is a core value, not a pair of a value and a runtime type tag. All four ghost maps and the state interpretation remain the same. The proof begins by unfolding `generic_wp`; thereafter the usual allocation, projection, load, bind, and frame rules apply. Erasure removes annotations, not the memory accesses that need justification. For the examples below, D, F contain the declarations of `genericId`, `Pair`, and `Box`; we omit these subscripts when they are fixed. Source-like calls in formulas abbreviate the corresponding explicit `gexpr` constructors.

Following a simple call. For `genericId<Int64>(7)`, take $\Phi(w) = \ulcorner w = 7 \urcorner$. Here and below, numerals abbreviate core integer values, and the corners $\ulcorner \cdot \urcorner$ embed an ordinary proposition into Iris. The proof establishes

$$\top \vdash \mathsf{gwp}\,(\mathsf{genericId}\langle \mathsf{Int64} \rangle(7))\,\{w.\ \ulcorner w = 7 \urcorner\}.$$

No memory ownership is required for this call. Its source expression is

```
GECall "genericId" [GTInt] [GEVal (LitV (LitInt 7))]
```

Erasure resolves `genericId` to its linked function value and places the argument in the internal one-element tuple. Execution evaluates that tuple, enters the function, projects argument zero into `x`, and returns `x`. None of these steps reads or allocates a heap cell. They are the same pure steps already used for ordinary calls. Replacing the source instance by `genericId<Bool>(true)` uses the same function implementation and proves that its result is `true`.

A type annotation and a representation predicate. The source annotation `Int64` tells the native compiler which type instance is intended. It does not say that the value is 7, positive, or equal to some earlier result. Likewise, `Box<Int64>` does not by itself supply a permission to read a particular object's field in Iris. Those facts belong in the specification.

We express them with a predicate $R : \mathsf{val} \to \mathsf{iProp}\,\Sigma$. For example, $R_7(w) = \ulcorner w = 7 \urcorner$ describes precisely the integer 7, whereas $R_{\geq 0}(w) = \exists n.\ \ulcorner w = n\ \wedge\ 0 \leq n \urcorner$ describes a nonnegative integer. Both can be useful for the same source type `Int64`. The implementation does not automatically derive R from a type annotation: the client chooses a predicate appropriate to the property being proved.

The uniform identity theorem, `identity_spec`, says that for every R, every source type argument t, and every input value v,

$$R(v) \vdash \mathsf{gwp}\,(\mathsf{genericId}\langle t \rangle(v))\,\{w.\ \ulcorner w = v \urcorner * R(w)\}.$$

Its English reading is: give identity an input with its representation assertion; get exactly that input back, with the same assertion. Taking $R = R_7$ gives the integer example above. Taking $R(w) = \ulcorner w = \mathsf{true} \urcorner$ gives the Boolean example. Taking $R = R_{\geq 0}$ preserves nonnegativity for any qualifying input. The proof works because identity never inspects or changes the input. The generic declaration alone does not prove this contract; the body and its Iris proof are essential. Section 9 displays the checked proof script.

Using the same theorem with an owned box. Now let the input be an object reference. Define

$$R_{\mathsf{box7}}(p) \triangleq \exists o.\ \ulcorner p = \mathsf{obj}(o) \urcorner * o \mapsto_o [0]7.$$

This says that p names an object and that we own its field zero, which contains 7. Unlike R_7, it includes a memory resource. Instantiate the identity theorem with $t = \mathsf{Box}\langle \mathsf{Int64} \rangle$ and $R = R_{\mathsf{box7}}$. Its postcondition then gives the same object reference and the same field ownership. Identity neither allocates a fresh box nor copies its contents.

This explains the proof of a small client:

```
let b = Box<Int64>(7)
let same = genericId<Box<Int64>>(b)
same.get()
```

Construction creates a fresh object o and supplies $o \mapsto_o [0]7$. The identity contract carries that assertion to `same` and establishes that `same` equals `b`. The `get` contract uses the field assertion to justify the read and returns 7 together with the field assertion. These are applications of `box_construct_spec`, `identity_spec`, and `box_get_spec`; composing them uses the bind rule in the next subsection. The type argument determines the native instance, while the points-to assertion explains why the memory access is safe.

Why an alias is not another ownership permission. After that call, both `b` and `same` name the same object. There are two names, but only one field-ownership resource. The contract returns $R_{\mathsf{box7}}(\mathit{same})$ and an equality allowing us to use either name. It does not return

$$R_{\mathsf{box7}}(b) * R_{\mathsf{box7}}(\mathit{same}).$$

Under the equality, this would require two full permissions for field zero of the same object, which the ghost-map algebra forbids. One may read the field through `b`, recover the permission, and then read through `same`; sequential reuse is different from having two independent permissions. In the proof, returning a resource means preserving the right to use it. It is not an extra operation performed by the program.

Pure assertions such as R_7 can be duplicated. An arbitrary predicate R need not have that property, so a theorem quantified over all R must not silently assume it. This is why the identity postcondition contains a pure equality and one copy of R.

Changing the result type changes the callback obligation. A slightly richer client is

```
let b = Box<Int64>(7)
let c = b.map<Bool>({ x: Int64 => 0 <= x })
c.get()
```

The receiver fixes `T = Int64`; the method chooses `U = Bool`. For this call, choose an input predicate $R(a) = \ulcorner a = 7 \urcorner$ and an output predicate $S(w) = \ulcorner w = \mathsf{true} \urcorner$. The callback obligation is

$$\forall a.\ R(a) \mathrel{-\!\!*} \mathsf{wp}\,(f\ a)\,\{S\}.$$

The wand $\mathrel{-\!\!*}$ means that supplying $R(a)$ is enough to establish the callback's weakest precondition. Here that proof is just the arithmetic fact $0 \leq 7$; the callback is also closed, as required by `box_map_spec`.

The method's proof reads the old field, uses the callback contract to obtain S, and puts the resulting Boolean into a newly allocated box. It preserves the old box's field ownership while the element assertion passes through the callback. In this example the element assertions are pure, so retaining them is harmless. In the general map theorem, R can own memory and the callback can consume it: the postcondition retains the old field and the new box's S, without promising another copy of the old R. Section 9.5 gives the precise contract and proof. The point of the two predicates is to describe this change in knowledge and ownership, not merely the change from the type name `Int64` to `Bool`.

What type-instantiation invariance buys us. For fixed D, F and a fixed postcondition Φ, the lemma `generic_wp_instantiate` states

$$\mathsf{gwp}_{D,F}\,(\mathsf{ginstantiate}\ s\ e)\,\{\Phi\} \ \dashv\vdash\ \mathsf{gwp}_{D,F}\,e\,\{\Phi\}.$$

The symbol $\dashv\vdash$ means that either assertion entails the other. The proof is simply to unfold `generic_wp` and rewrite by `erase_instantiate`: both sides ask about exactly the same core expression and postcondition.

For example, start with the source call `GECall "genericId" [GTVar "T"] [GEVar "x"]`. Instantiating `T` changes its type argument but leaves its erased call and the term variable `x` unchanged. The hypothesis `nominal_owners e = true` similarly ensures that a receiver annotation such as `Box<T>` keeps its outer layout name `Box`. Substitution may change the element type; it cannot turn an object allocation into struct construction. A bare type variable in place of the layout name would not meet this condition.

This law removes a repeated proof obligation about type annotations. It does not identify calls with different input data: `genericId<Int64>(7)` and `genericId<Bool>(true)` have different erased arguments and different results. They share an implementation and an instance of the uniform identity theorem. Similarly, instantiation does not automatically replace one chosen representation predicate by another; the theorem must quantify over the predicates needed by the client.

Why old proofs still apply. The embedding lemma `generic_wp_legacy` is unconditional:

$$\mathsf{gwp}_{D,F}\,(\mathsf{GECore}\;e)\,\{\Phi\} \;\dashv\vdash\; \mathsf{wp}\,e\,\{\Phi\}.$$

`GECore` wraps an old core expression, and erasure returns that expression exactly. For example, an existing proof that a core addition of 2 and 3 returns 5 is immediately a proof of its `GECore` embedding with the same postcondition. There are no new heap resources or extra type premises to supply. The embedding is an interface for logical reuse; its extraction continues to use the legacy printer.

Thus a generic proof has a concrete task: show that the erased computation uses the supplied resources safely and establishes the stated result. Well-formed annotations help construct and print the source, but are not a substitute for that proof. Adequacy connects the proved weakest precondition to safe execution in Section 7.4.

6.4 Structural rules

These three rules are why the logic is a separation logic rather than a Hoare logic with an explicit frame side-condition. Bind and the pure-step lemma are proved in `primitive_laws.v` by appealing to Iris. The frame rule is Iris's own `wp_frame_l`; this development does not re-prove it.

```
Lemma wp_bind K e Phi :
  wp e (fun v => wp (fill K (Val v)) Phi) |- wp (fill K e) Phi.
```

Lemma 3 (Bind)**.** *If K is an evaluation context, then*

$$\mathsf{wp}\,e\,\{\,v.\;\mathsf{wp}\,K[v]\,\{\Phi\}\,\}\;\vdash\;\mathsf{wp}\,K[e]\,\{\Phi\}.$$

Intuition. To prove a specification of a term with a hole, we prove a specification of the hole, and we allow the postcondition of the hole to talk about the rest of the program. Left-to-right evaluation is just a choice of K. No memory fact has to be mentioned. Facts that the hole does not use stay outside this rule and are framed.

Lemma 4 (Frame)**.**

$$P * \mathsf{wp}\,e\,\{\Phi\}\;\vdash\;\mathsf{wp}\,e\,\{\,v.\;P * \Phi(v)\,\}.$$

Intuition. If a proof of e never mentions P, then e cannot depend on the cells inside P, and those cells survive e unchanged. This is the rule that makes "the other pointer is untouched" a consequence of not putting that pointer into the callee's precondition. It is not proved by induction on CJR expressions.

Lemma 5 (Consequence)**.**

$$\mathsf{wp}\, e\, \{\Phi\} \;\wedge\; (\forall v.\, \Phi(v) \mathrel{-\!\!*} \Psi(v)) \;\vdash\; \mathsf{wp}\, e\, \{\Psi\}.$$

Intuition. A stronger postcondition can be weakened. The examples use this when a lemma such as `bump_wp` returns both a pure result and a framed cell, and the continuation only needs one of them. In Coq it is `wp_wand` or `wp_mono`. The same rules apply to `gwp` by unfolding erasure. For example, $P * \mathsf{gwp}\, e\, \{\Phi\} \vdash \mathsf{gwp}\, e\, \{v.\, P * \Phi(v)\}$. A callback specification must supply every resource it uses; the caller can frame unrelated resources, including a box's own field, around that call.

6.5 Pure rules

A pure head step is one that is defined in every state, does not change the state, and has exactly one successor. The general-purpose lemma is `wp_pure_step`. Each concrete rule below is an instance. Where the step produces a value immediately, the postcondition is under a later, because that is how this lemma returns the credit. Where the step produces another expression, the rule simply asks for a weakest precondition of that expression.

Closures and application.

```
Lemma wp_rec f x e Phi :
  later (pound 1 -*- Phi (RecV f x e)) |- wp (Rec f x e) Phi.
Lemma wp_app f x e v Phi :
  wp (subst_binder x v (subst_binder f (RecV f x e) e)) Phi
    |- wp (App (Val (RecV f x e)) (Val v)) Phi.
```

$$\triangleright(\Phi(\mathsf{rec}\, f\, x.\, e)) \vdash \mathsf{wp}\, (\mathsf{rec}\, f\, x.\, e)\, \{\Phi\},$$
$$\mathsf{wp}\, e[x \mapsto v][f \mapsto \mathsf{rec}\, f\, x.\, e]\, \{\Phi\} \vdash \mathsf{wp}\, ((\mathsf{rec}\, f\, x.\, e)\, v)\, \{\Phi\}.$$

Intuition. Evaluating a function literal yields the closure as a value. Calling it substitutes the argument and, if the function is recursive, the closure itself. No heap fact is required: the code is data, and the data is the value in hand. That is what lets a vtable be an ordinary struct of closures.

Let, if, while, sequencing, arithmetic, offset.

$$\mathsf{wp}\, e[x \mapsto v]\, \{\Phi\} \vdash \mathsf{wp}\, (\mathsf{let}\, x = v\, \mathsf{in}\, e)\, \{\Phi\}$$
$$\mathsf{wp}\, e_1\, \{\Phi\} \vdash \mathsf{wp}\, (\mathsf{if}\ \mathsf{true}\ \mathsf{then}\ e_1\ \mathsf{else}\ e_2)\, \{\Phi\}$$
$$\mathsf{wp}\, e_2\, \{\Phi\} \vdash \mathsf{wp}\, (\mathsf{if}\ \mathsf{false}\ \mathsf{then}\ e_1\ \mathsf{else}\ e_2)\, \{\Phi\}$$
$$\mathsf{wp}\, (\mathsf{if}\ e_1\ \mathsf{then}\ (e_2;\, \mathsf{while}\, e_1\, \mathsf{do}\, e_2)\ \mathsf{else}\ ())\, \{\Phi\} \vdash \mathsf{wp}\, (\mathsf{while}\, e_1\, \mathsf{do}\, e_2)\, \{\Phi\}$$
$$\mathsf{wp}\, e\, \{\Phi\} \vdash \mathsf{wp}\, (v; e)\, \{\Phi\}$$
$$[\![\oplus]\!](v_1, v_2) = \mathsf{Some}(v) \Rightarrow \triangleright(\Phi(v)) \vdash \mathsf{wp}\, (v_1 \oplus v_2)\, \{\Phi\}$$
$$\triangleright(\Phi(\mathsf{ptr}(\ell + i))) \vdash \mathsf{wp}\, (\mathsf{ptr}(\ell) + i)\, \{\Phi\}.$$

Multiplication is also a pure binary step: for integer operands a, b, `MulOp` returns the unbounded product $a \cdot b$. The specialized rule `wp_mul` applies `wp_binop` and needs no numeric side condition.

Printing it as Cangjie `Int64` multiplication does not insert a runtime range check or prove absence of overflow. The hash-map example supplies its own bound for the reduced-key product.

The specialized remainder rule makes its domain explicit:

```
Lemma wp_rem (n d : Z) E Phi :
  d <> 0 ->
  later (pound 1 -*- Phi (LitV (LitInt (Z.rem n d))))
  |- WP BinOp RemOp (Val (LitV (LitInt n)))
                  (Val (LitV (LitInt d))) @ E {{ Phi }}.
```

Its proof specializes `wp_binop`; no new ghost state or ownership rule is needed. The printer emits Cangjie's `%` operator. The nonzero premise excludes a stuck zero-divisor operation in CJR; the language does not yet model the target's arithmetic exceptions.

Bitwise rules. The integer XOR and AND rules also specialize `wp_binop`:

```
Lemma wp_bitxor (a b : Z) E Phi :
  later (pound 1 -*- Phi (LitV (LitInt (Z.lxor a b))))
  |- WP BinOp BitXorOp (Val (LitV (LitInt a)))
                       (Val (LitV (LitInt b))) @ E {{ Phi }}.
Lemma wp_bitand (a b : Z) E Phi :
  later (pound 1 -*- Phi (LitV (LitInt (Z.land a b))))
  |- WP BinOp BitAndOp (Val (LitV (LitInt a)))
                       (Val (LitV (LitInt b))) @ E {{ Phi }}.
```

Neither needs an additional numeric side condition: both operations are defined for every pair of integers. As with the remainder rule, the `later` and step credit account for the pure reduction, and no heap ownership changes. Byte bounds used in AES are established by the example's pure lemmas rather than by the primitive rules.

Intuition for `while`. The loop rule does not mention an invariant; it unfolds the loop once into a conditional. The invariant is whatever assertion one has in hand when returning to the same syntactic loop. If the condition is false, the unfolding takes the else branch and returns unit. If it is true, the body runs and the same rule applies again. Termination is not proved; the unfolding is used only as many times as the concrete execution needs.

Intuition for offset. Adding an index to a pointer is arithmetic. It does not check that the resulting address lies in a block. The check happens when someone loads or stores: without a points-to fact at the computed address, the load rule does not apply, and the machine would be stuck. Bounds are therefore a matter of which points-to facts the proof owns, not of a side condition on $+$.

Struct values.

$$\triangleright(\Phi(\langle \overline{v} \rangle)) \vdash \mathsf{wp}\, \langle \overline{v} \rangle\, \{\Phi\},$$
$$\overline{v}(i) = v \Rightarrow \triangleright(\Phi(v)) \vdash \mathsf{wp}\, (\langle \overline{v} \rangle.i)\, \{\Phi\}.$$

Intuition. A finished zipper is the unboxed tuple. Reading a component copies it out. The tuple is not a heap cell, so there is nothing to frame and nothing to free. If the index is out of range, there is no rule.

Building a struct or an object from a non-empty expression list is also a pure step (`StructStepS`, `NewStepS`). The development uses `wp_pure_step` for that step in the vtable example rather than a named lemma.

6.6 Stack rules

```
Lemma wp_var_bind x v e Phi :
  (forall l, StackId l |->s v -*-
     wp (subst_var x l e)
        (fun w => exists v', StackId l |->s v' * Phi w))
  |- wp (VarBind x (Val v) e) Phi.

Lemma wp_stack_load l v Phi :
  l |->s v -*- later (pound 1 -*- l |->s v -*- Phi v)
    |- wp (StackLoad (stk l)) Phi.

Lemma wp_stack_assign l v w Phi :
  l |->s v -*- later (pound 1 -*- l |->s w -*- Phi unit)
    |- wp (StackAssign (stk l) (Val w)) Phi.

Lemma wp_stack_field l vs i w v Phi :
  vs !! i = Some w ->
  l |->s StructV vs -*-
    later (pound 1 -*- l |->s StructV (set_nth i v vs) -*- Phi unit)
    |- wp (StackFieldStore (stk l) i (Val v)) Phi.
```

In mathematics, with ℓ a stack address:

$$\big(\forall \ell.\ \ell \mapsto_s v \mathrel{-\!\!*} \mathsf{wp}\, e\{x \mapsto \ell\}\,\{\, w.\ \exists v'.\ \ell \mapsto_s v' * \Phi(w)\,\}\big) \vdash \mathsf{wp}\,(\mathsf{var}\ x = v\ \mathsf{in}\ e)\,\{\Phi\},$$
$$\ell \mapsto_s v \mathrel{-\!\!*} \triangleright(\ell \mapsto_s v \mathrel{-\!\!*} \Phi(v)) \vdash \mathsf{wp}\,(\mathsf{stk}(\ell))\,\{\Phi\},$$
$$\ell \mapsto_s v \mathrel{-\!\!*} \triangleright(\ell \mapsto_s w \mathrel{-\!\!*} \Phi(())) \vdash \mathsf{wp}\,(\mathsf{stk}(\ell) := w)\,\{\Phi\},$$
$$\overline{v}(i) = w \Rightarrow \ell \mapsto_s \langle \overline{v} \rangle \mathrel{-\!\!*} \triangleright\big(\ell \mapsto_s \langle \overline{v}[i \mapsto u] \rangle \mathrel{-\!\!*} \Phi(())\big) \vdash \mathsf{wp}\,(\mathsf{stk}(\ell).i := u)\,\{\Phi\}.$$

Intuition for ***var****.* The rule allocates a fresh slot, hands exclusive ownership of it to the proof, and asks for a proof of the body after the name has been compiled into stack operations. The contents may be changed. At the end the slot must be given back, at whatever value it then has. The pop administrative step deletes it, so the caller does not see a stack fact. This is lexical scope: the owner of the cell is the body, and the cell does not escape.

The existential is essential. If the postcondition demanded the original value, an assignment inside the body could not be verified, because assignment replaces the points-to fact.

Intuition for load, assign, and field update. Load reads and returns the same fact. Assignment exchanges the fact for one about the new value. Field update is assignment of a struct, with one component changed, and it is only available when that component already exists. A missing component is stuck, and there is no points-to fact that would let the rule fire. None of these rules allocates on the raw heap or the object heap.

6.7 Raw-heap rules

```
Lemma wp_alloc_one Phi :
  (forall l, RawId l |->r LitV (LitInt 0) -*-
             RawId l |->b 1 -*- Phi (LitV (LitPtr l)))
    |- wp (Alloc (Val (LitV (LitInt 1)))) Phi.

Lemma wp_free_one l v Phi :
```

```
  RawId l |->r v -*- RawId l |->b 1 -*- later (Phi unit)
    |- wp (Free (Val (LitV (LitPtr l)))) Phi.

Lemma wp_load l v Phi :
  RawId l |->r v -*- later (RawId l |->r v -*- Phi v)
    |- wp (Load (Val (LitV (LitPtr l)))) Phi.

Lemma wp_store l v w Phi :
  RawId l |->r v -*- later (RawId l |->r w -*- Phi unit)
    |- wp (Store (Val (LitV (LitPtr l))) (Val w)) Phi.
```

$$\big(\forall \ell.\ \ell \mapsto_r 0 * \ell \mapsto_b 1 \mathrel{-\!\!*} \Phi(\mathsf{ptr}(\ell))\big) \vdash \mathsf{wp}\,(\mathsf{alloc}(1))\,\{\Phi\},$$
$$\ell \mapsto_r v * \ell \mapsto_b 1 \mathrel{-\!\!*} \triangleright(\Phi(())) \vdash \mathsf{wp}\,(\mathsf{free}(\ell))\,\{\Phi\},$$
$$\ell \mapsto_r v \mathrel{-\!\!*} \triangleright(\ell \mapsto_r v \mathrel{-\!\!*} \Phi(v)) \vdash \mathsf{wp}\,(!\,\ell)\,\{\Phi\},$$
$$\ell \mapsto_r v \mathrel{-\!\!*} \triangleright(\ell \mapsto_r w \mathrel{-\!\!*} \Phi(())) \vdash \mathsf{wp}\,(\ell := w)\,\{\Phi\}.$$

Intuition. Allocation creates a cell that was not owned before, initialised to zero, together with a separate fact that a block of length one starts there. The two facts have different jobs. The points-to fact lets one load and store the word. The block fact lets one free the block. Free consumes both, so the word cannot be used again: there is no fact left to feed to the load rule, and the concrete word has been deleted, so the machine would get stuck if one tried.

Load does not disturb the cell. Store keeps ownership and changes the value. Neither rule needs the block fact. A client may therefore load and store while the right to free sits unused in the context, and the frame rule keeps it there.

The operational rule for allocation accepts any positive length. The verified rule above is the one-word instance, which the small examples use. The array library uses the general rule: a points-to fact per word, plus one block fact for the base and the length.

6.8 Object rules

```
Lemma wp_new vs Phi :
  (forall o, ([* list] i |-> v in vs, ObjId o |->o[i] v) -*-
             Phi (LitV (LitObj o)))
    |- wp (New vs []) Phi.

Lemma wp_field_load o i v Phi :
  ObjId o |->o[i] v -*- later (ObjId o |->o[i] v -*- Phi v)
    |- wp (FieldLoad (Val (LitV (LitObj o))) i) Phi.

Lemma wp_field_store o i v w Phi :
  ObjId o |->o[i] v -*- later (ObjId o |->o[i] w -*- Phi unit)
    |- wp (FieldStore (Val (LitV (LitObj o))) i (Val w)) Phi.
```

$$\big(\forall o.\ \mathop{*}_{i<k} o \mapsto_o [i]v_i \mathrel{-\!\!*} \Phi(\mathsf{obj}(o))\big) \vdash \mathsf{wp}\,(\mathsf{new}(v_0, \ldots, v_{k-1}))\,\{\Phi\},$$
$$o \mapsto_o [i]v \mathrel{-\!\!*} \triangleright(o \mapsto_o [i]v \mathrel{-\!\!*} \Phi(v)) \vdash \mathsf{wp}\,(o.i)\,\{\Phi\},$$
$$o \mapsto_o [i]v \mathrel{-\!\!*} \triangleright(o \mapsto_o [i]w \mathrel{-\!\!*} \Phi(())) \vdash \mathsf{wp}\,(o.i := w)\,\{\Phi\}.$$

Intuition. `new` of a finished argument list allocates a fresh object and returns one points-to fact per field, with the initialising values. There is no block fact and no free rule: the object stays allocated. Field load and field store are the same idea as raw load and store, on a different map. Owning $o \mapsto_o [i]v$ says nothing about $o \mapsto_o [j]w$ for $j \neq i$, and nothing about any raw word. That is why a raw cell can be framed across a field update, and why two fields of one object can be reasoned about separately.

6.9 Exclusivity

```
Lemma stack_mapsto_exclusive l v1 v2 :
  l |->s v1 -*- l |->s v2 |- False.
Lemma raw_mapsto_exclusive l v1 v2 :
  l |->r v1 -*- l |->r v2 |- False.
```

Lemma 6 (Exclusive ownership)**.** *For a stack address or a raw address ℓ, and any values v_1, v_2,*

$$\ell \mapsto_s v_1 * \ell \mapsto_s v_2 \vdash \bot, \qquad \ell \mapsto_r v_1 * \ell \mapsto_r v_2 \vdash \bot.$$

Intuition. Full ownership is unique. The ghost-map algebra rejects two fragments of fraction 1 at the same key, even if they agree on the value. This is the specification of non-aliasing. If a swap is specified with two points-to assumptions, applying it to one address twice is impossible: the precondition is false. The machine might still have a single cell there; the logic refuses to pretend there are two.

The same algebra applies to block facts and to object fields. The development records the stack and raw cases because those are the ones the examples need to point at. It also constrains generic representations. Copying a `Pair` value or returning the element of a `Box` does not duplicate resources owned through the values it contains. In particular, a map callback may consume an element predicate even though the original box still stores the same runtime reference. Its postcondition must record that distinction.

7 Main theorems

7.1 Head reduction is deterministic

Theorem 1 above is `base_step_det`. Restated: the head reduction relation is a partial function of (e, σ). Combined with the context machinery, a configuration has at most one successor. Stuckness is therefore unambiguous. Either the term is a value, or exactly one head redex is ready and the rule applies, or no rule applies and the program is stuck.

7.2 Well-formedness is an invariant of the verified steps

The state interpretation contains $\ulcorner \mathsf{wf}(\sigma) \urcorner$. Each memory rule proves that the state after the step is well formed if the state before was. The statements are `wf_stack_alloc`, `wf_stack_delete`, `wf_stack_upd`, `wf_raw_store`, `wf_alloc_one`, `wf_free_one`, `wf_new`, and `wf_field_store`.

Lemma 7 (Allocator invariant)**.** $\mathsf{wf}(\sigma_0)$*. If* $\mathsf{wf}(\sigma)$ *and a verified memory rule steps to* σ'*, then* $\mathsf{wf}(\sigma')$*.*

The one-word cases are the ones used by the logic. For example, freeing a block of length 1 at ℓ preserves coverage of every other block because disjointness puts their words outside $\{\ell\}$.

7.3 Adequacy

```
Lemma cjr_adequacy S e (phi : val -> Prop) :
  (forall `{!cjrGS S} `{!invGS_gen HasLc S},
     |- wp e (fun v => pure (phi v))) ->
  adequate NotStuck e state_init (fun v _ => phi v).
```

Theorem 8 (Adequacy)**.** *Let φ be a pure predicate on values. Suppose that in every Iris instantiation of the CJR ghost state one can prove*

$$\mathsf{wp}\, e\, \{\, v.\, \ulcorner \varphi(v) \urcorner \,\}.$$

Then, executing e from σ_0, every reachable configuration can take a step or is a value, and every final value v satisfies $\varphi(v)$.

What the proof does. It allocates four empty ghost maps, packages them as the CJR ghost state, and observes $\mathsf{wf}(\sigma_0)$. Iris's adequacy theorem for weakest preconditions then applies. The hypothesis is a proof that does not depend on a particular ghost name, so it can be used at the names chosen here.

What it does not say. It does not say that every closed expression is safe. An expression with a free variable, a null dereference, or a load of an address nobody allocated has no such proof, and the theorem is silent. It also does not extract a functional-correctness result beyond the pure φ that was proved. If the postcondition is only "the result is unit", adequacy says the program returns unit without getting stuck; the interesting memory facts were used and consumed inside the proof.

The closed examples `sum_one`, `bump_keep`, `cell`, `class_bump`, and `vtable_prog` are instantiated at this theorem. The other examples are open: their hypotheses are points-to facts, so they are stated as weakest preconditions rather than as runs from σ_0.

7.4 Generic adequacy and conservative extension

For a generic program p, let $D = \mathsf{gp_layouts}(p)$, $F = \mathsf{link_functions}\ D\ []\ (\mathsf{gp_functions}(p))$, and $e = \mathsf{gp_main}(p)$. The theorem `generic_adequacy` says that a proof of $\mathsf{gwp}_{D,F}\, e\, \{v.\, \ulcorner \varphi(v) \urcorner\}$ in every Iris instantiation implies

$$\mathsf{adequate}\ \mathsf{NotStuck}\ (\mathsf{erase_program}\ p)\ \sigma_0\ (\lambda v\ _.\ \varphi(v)).$$

Its proof unfolds `generic_wp` and applies `cjr_adequacy`. For the closed identity client, `generic_main_adequate` establishes that the erased call `genericId<Int64>(42)` is safe and every final value is `LitV (LitInt 42)`. The native program's `main` separately returns exit status zero after evaluating that call.

Together, `erase_legacy` and `generic_wp_legacy` establish conservative inclusion of the original operational language and logic. Existing proofs and monomorphic extraction interfaces are retained. The structural source checker alone is not a safety proof: adequacy still requires a weakest-precondition proof. Nor does this theorem establish preservation by the printer or the native compiler.

8 The examples

Each example uses a feature that the previous one does not. The files appear below in tutorial order. The programs are written in the Cangjie fragment above. The elementary examples in this

section prove core `expr` terms. Section 9 follows them with generic `gexpr` definitions and proofs about their erasures, before the larger data structures.

8.1 Arithmetic: `arith.v`

Program.

```
// Generated by tools/extraction/rocq/emit_cj.v from the CJR core terms.

foreign func calloc(count: UIntNative, size: UIntNative): CPointer<Unit>
foreign func free(ptr: CPointer<Unit>): Unit

func sumOne(): Int64 {
    var n = 1
    var s = 0
    while (1 <= n) {
        s = s + 1
        n = n - 1
    }
    s
}

func multiply(a: Int64, b: Int64): Int64 {
    a * b
}

func divide(a: Int64, b: Int64): Int64 {
    a / b
}

main(): Unit {
    sumOne()
}
```

Theorem.

$$\mathsf{wp}\,\mathsf{sum_one}\,\{\, v.\, \ulcorner v = 1 \urcorner \,\}.$$

From σ_0, the program returns 1 and does not get stuck (`sum_one_adequate`).

What is new. Nothing is allocated on either heap. The proof uses `var`, stack load and assignment, `while`, `if`, sequencing, and integer $+$ and $-$. The loop condition is true once: n is 1, then 0. On the true branch, s becomes 1 and n becomes 0. On the false branch the loop returns unit, and the trailing read returns s.

The stack facts at the end are $s \mapsto_s 1$ and $n \mapsto_s 0$. They are given back to `wp_var_bind` through the existential, and `pop` deletes them. The pure postcondition does not mention the stack. That is the scope rule from the previous section, exercised on a loop that really mutates.

8.2 Stack structs: `struct_upd.v`

Program.

```
// Generated by tools/extraction/rocq/emit_cj.v from the CJR core terms.

foreign func calloc(count: UIntNative, size: UIntNative): CPointer<Unit>
foreign func free(ptr: CPointer<Unit>): Unit

struct Cell {
    public var x: Int64
    public init(x: Int64) {
        this.x = x
    }
}

func bump(): Int64 {
    var p = Cell(0)
    p.x = 1
    p.x
}

func bumpKeep(): Int64 {
    let r = unsafe { CPointer<Int64>(calloc(UIntNative(1), UIntNative(8))) }
    bump()
    unsafe { r.read() }
}

main(): Unit {
    bumpKeep()
}
```

Theorems.

$$\ell \mapsto_r v \vdash \mathsf{wp}\,\mathsf{bump}\,\{\, w.\ \ulcorner w = 1 \urcorner * \ell \mapsto_r v \,\},$$
$$\mathsf{wp}\,\mathsf{bump_keep}\,\{\, v.\ \ulcorner v = 0 \urcorner \,\}.$$

The closed run from σ_0 returns 0 (`bump_adequate`).

What is new. The struct lives in a stack slot, not on the raw heap. The field write is `wp_stack_field`: the slot changes from $\langle 0 \rangle$ to $\langle 1 \rangle$, and the read copies the component out with `wp_struct_load`. No raw address is allocated by bump itself.

The first theorem is the frame. Its hypothesis is an arbitrary raw cell, and the postcondition returns that same cell. The proof of the stack update never mentions it, so it cannot have been changed. The closed program makes the same point operationally: the fresh raw word is 0 before bump and 0 after, which is what the final load returns. The field update's result 1 is established on the way and then discarded by sequencing, because the observation we keep is the unchanged raw word.

8.3 One raw cell: `cell.v`

Program.

```
// Generated by tools/extraction/rocq/emit_cj.v from the CJR core terms.

foreign func calloc(count: UIntNative, size: UIntNative): CPointer<Unit>
```

```
foreign func free(ptr: CPointer<Unit>): Unit

func cell(): Unit {
    let p = unsafe { CPointer<Int64>(calloc(UIntNative(1), UIntNative(8))) }
    unsafe { p.write(7) }
    unsafe { p.read() }
    unsafe { free(CPointer<Unit>(p)) }
}

main(): Unit {
    cell()
}
```

Theorem.

$$\mathsf{wp}\,\mathsf{cell}\,\{\,v.\,\ulcorner v = ()\urcorner\,\}.$$

From σ_0, the program returns unit and does not get stuck.

What is new. This is the first use of $\mapsto_r$ and $\mapsto_b$. Allocation produces $\ell \mapsto_r 0$ and $\ell \mapsto_b 1$. The store replaces the points-to fact by $\ell \mapsto_r 7$. The load reads 7 and returns the same fact. The program then discards that 7 by sequencing and frees the block. Free consumes both facts and returns unit.

The pure specification does not say "the loaded value was 7"; that fact is internal to the proof. What adequacy reports is that the whole expression, whose value is the value of `free`, returns unit without a stuck load, store, or double free. After `free`, no points-to fact remains, so a further load could not be verified.

8.4 Swap and aliasing: `swap.v`

Program.

```
// Generated by tools/extraction/rocq/emit_cj.v from the CJR core terms.

foreign func calloc(count: UIntNative, size: UIntNative): CPointer<Unit>
foreign func free(ptr: CPointer<Unit>): Unit

func swap(l1: CPointer<Int64>, l2: CPointer<Int64>): Unit {
    let t = unsafe { l1.read() }
    unsafe { l1.write(unsafe { l2.read() }) }
    unsafe { l2.write(t) }
}

main(): Unit {
    let l1 = unsafe { CPointer<Int64>(calloc(UIntNative(1), UIntNative(8))) }
    let l2 = unsafe { CPointer<Int64>(calloc(UIntNative(1), UIntNative(8))) }
    unsafe { l1.write(1) }
    unsafe { l2.write(2) }
    swap(l1, l2)
}
```

Theorems.

$$\ell_1 \mapsto_r v_1 * \ell_2 \mapsto_r v_2 \vdash \mathsf{wp}\,\mathsf{swap}(\ell_1, \ell_2)\,\{\,_.\ \ell_1 \mapsto_r v_2 * \ell_2 \mapsto_r v_1\,\},$$
$$\ell \mapsto_r v_1 * \ell \mapsto_r v_2 \vdash \bot.$$

What is new. The postcondition needs both cells. The proof loads ℓ_2 while still owning ℓ_1, stores into ℓ_1, then stores the saved value into ℓ_2. Each rule mentions only the cell it touches; the other fact sits in the context and is framed.

The second lemma is `raw_mapsto_exclusive`, re-exported as `swap_aliased`. It is not a second proof of the same swap. It says that the swap specification cannot be instantiated at one address twice: the hypothesis of two points-to facts at that address is absurd. An overlapping specification would be a different theorem—for example a single cell updated twice—and it is not claimed here.

8.5 Indexed blocks: `sum_block.v`

Program.

```
// Generated by tools/extraction/rocq/emit_cj.v from the CJR core terms.

foreign func calloc(count: UIntNative, size: UIntNative): CPointer<Unit>
foreign func free(ptr: CPointer<Unit>): Unit

func sumAt(p: CPointer<Int64>): Int64 {
    var i = 0
    var s = 0
    while (i <= 0) {
        s = s + unsafe { (p + i).read() }
        i = i + 1
    }
    s
}

main(): Unit {
    let p = unsafe { CPointer<Int64>(calloc(UIntNative(1), UIntNative(8))) }
    unsafe { p.write(4) }
    sumAt(p)
}
```

Theorem.

$$p \mapsto_r 4 * p \mapsto_b 1 \vdash \mathsf{wp}\,\mathsf{sum_at}(p)\,\left\{\,v.\ \ulcorner v = 4 \urcorner * p \mapsto_r 4 * p \mapsto_b 1\,\right\}.$$

What is new. The address that is loaded is not the base written in the source; it is the value of Offset, that is $p + i$. On the only iteration, i is 0, and $p + 0 = p$, so the load is justified by the points-to fact at p. The addition itself is the pure offset rule; it does not consume the points-to fact. The loop then increments i to 1, the condition $i \leq 0$ fails, and the sum 4 is returned.

The block-length fact is never used by the load, and it is still there at the end. That is the split between "I may read this word" and "I may free this block". The example does not free. A load of $p + 1$ would not be provable from these hypotheses: there is no points-to fact at $p + 1$, and a one-word block does not contain that word.

The loop invariant, in the ordinary sense, is the assertion carried across the unfolding: i and s are stack cells, s holds the sum of the prefix already visited, and the raw word is unchanged. For a one-iteration loop the prefix is either empty or the single element 4.

8.6 Final classes: `class_upd.v`

Program.

```
// Generated by tools/extraction/rocq/emit_cj.v from the CJR core terms.

foreign func calloc(count: UIntNative, size: UIntNative): CPointer<Unit>
foreign func free(ptr: CPointer<Unit>): Unit

class Box {
    public var value: Int64
    public init(value: Int64) {
        this.value = value
    }
}

func classBump(): Int64 {
    let r = unsafe { CPointer<Int64>(calloc(UIntNative(1), UIntNative(8))) }
    let o = Box(0)
    o.value = 5
    o.value
    unsafe { r.read() }
}

main(): Unit {
    classBump()
}
```

Theorem.

$$\mathsf{wp}\,\mathsf{class_bump}\,\{\,v.\,\ulcorner v = 0\urcorner\,\}.$$

From σ_0, the program returns 0 and does not get stuck.

What is new. `new` allocates on the object map, not the raw map. The client receives $o \mapsto_o [0]0$. The field store changes it to $o \mapsto_o [0]5$, and the field load reads 5. The raw word allocated first is 0, is never stored, and is what the program returns.

So the closed observation is again the framed cell: object update does not change the raw heap. The proof makes the separation concrete. After `wp_alloc_one`, the raw points-to fact stays in the context through `wp_new`, `wp_field_store`, and `wp_field_load`. None of those lemmas takes a raw fact as a hypothesis. The final `wp_load` is what consumes it.

8.7 Framing across a call: `frame_call.v`

Program.

```
// Generated by tools/extraction/rocq/emit_cj.v from the CJR core terms.
```

```
foreign func calloc(count: UIntNative, size: UIntNative): CPointer<Unit>
foreign func free(ptr: CPointer<Unit>): Unit

func write(x: CPointer<Int64>): Unit {
    unsafe { x.write(9) }
}

func frameCall(p: CPointer<Int64>, q: CPointer<Int64>): Int64 {
    write(p)
    unsafe { q.read() }
}

main(): Unit {
    let p = unsafe { CPointer<Int64>(calloc(UIntNative(1), UIntNative(8))) }
    let q = unsafe { CPointer<Int64>(calloc(UIntNative(1), UIntNative(8))) }
    unsafe { q.write(3) }
    frameCall(p, q)
}
```

Theorem.

$$p \mapsto_r v * q \mapsto_r 3 \vdash \mathsf{wp}\ \mathsf{write_call}(p, q) \left\{ w.\ \ulcorner w = 3 \urcorner * p \mapsto_r 9 * q \mapsto_r 3 \right\}.$$

What is new. The callee is a closure. `wp_rec` makes the closure, `wp_app` substitutes the pointer p for x, and `wp_store` needs only $p \mapsto_r v$. The fact $q \mapsto_r 3$ is not in the callee's precondition. It is framed across the call, then used by the load, which returns 3.

This is the frame rule applied at a function boundary. Nothing about `App` is special: substitution puts a value in the body, and the body is an ordinary store. The guarantee that q is unchanged is simply the guarantee that the store's proof did not receive q.

8.8 A hand-written vtable: `vtable.v`

Program.

```
// Generated by tools/extraction/rocq/emit_cj.v from the CJR core terms.

foreign func calloc(count: UIntNative, size: UIntNative): CPointer<Unit>
foreign func free(ptr: CPointer<Unit>): Unit

struct VTable {
    public var inc: (Counter) -> Unit
    public init(inc: (Counter) -> Unit) {
        this.inc = inc
    }
}

class Counter {
    public var value: Int64
    public init(value: Int64) {
        this.value = value
    }
}
```

```
func inc(self: Counter): Unit {
    self.value = self.value + 1
}

func run(): Int64 {
    let box = VTable(inc)
    let o = Counter(0)
    box.inc(o)
    o.value
}

main(): Unit {
    run()
}
```

Theorem.

$$\mathsf{wp}\ \mathsf{vtable_prog}\ \{\, v.\ulcorner v = 1\urcorner \,\}.$$

From σ_0, the program returns 1 and does not get stuck.

What is new. The function is a value, the struct is a tuple holding that value, and the call is an ordinary application of a component that was loaded out of the tuple. There is no method table in the machine and no dynamic dispatch rule. A "method" is a function whose first argument is the object, and a "vtable" is a struct the programmer assembled.

The proof follows that reading. First, the closure is formed by `wp_rec`. Second, the struct zipper evaluates the closure and becomes $\langle inc \rangle$ by the pure struct rules. `new` yields $o \mapsto_o [0]0$. `wp_struct_load` copies the closure out; this does not touch the object. `wp_app` substitutes the object reference into the body. The body loads field 0, adds one, and stores 1. Finally, the trailing field load reads that 1, which is the value of the whole program.

All three regions of the story meet: a stack-free unboxed struct of code, a code pointer, and an object field. The raw heap is not used. The earlier examples already showed that leaving it unused is safe.

9 Parametric polymorphism and conservative erasure

This section puts the source language of Section 3.5 and the logic of Section 6.3 to work. The checked definitions and proofs are in `example/generics/rocq/generic_examples.v`. The program defines `genericId<T>`, `Pair<A, B>`, and `Box<T>` with `get` and `map<U>`. The same definitions feed the generic printer. Each contract below is uniform in its element predicates and source type arguments; it is not a collection of integer-only proofs.

9.1 From a declaration to the program being proved

Identity is a particularly small example of a generic declaration:

```
Definition identity_body := GEVar "x".
Definition identity_fn :=
  GFunction "genericId" None ["T"]
    [("x", GTVar "T")] (GTVar "T") identity_body.
```

The declaration has no owner, binds one type parameter and one value parameter, and returns that value. Its source call is `GECall "genericId" [t] [GEVal v]`. Linking produces a recursive core value with an internal tuple parameter; its body projects argument zero and returns it. The annotations `T` and `t` are absent from that value. The printer instead retains them, producing

```
public func genericId<T>(x: T): T {
    return x
}
```

Thus there are two uses of one structured source: erasure supplies the program for the Iris proof, and printing supplies the native generic declaration. The mathematical notation gwp below fixes `generic_layouts` and `generic_runtime`; source-like calls inside it abbreviate their explicit `gexpr` constructors.

9.2 Identity: one proof for every representation

For every predicate $R : \mathsf{val} \to \mathsf{iProp}\,\Sigma$, type argument t, and value v, `identity_spec` establishes

$$R(v) \;\vdash\; \mathsf{gwp}\,(\mathsf{genericId}\langle t\rangle(v))\,\{w.\ulcorner w = v\urcorner * R(w)\}.$$

The result is the exact input value, and the client's representation resource is returned. No persistence assumption on R is used. The proof script is short enough to display in full:

```
Lemma identity_spec (R : val → iProp Σ) t v :
  R v -∗ generic_wp generic_layouts generic_runtime
    (GECall "genericId" [t] [GEVal v]) (λ w, ⌜w = v⌝ ∗ R w).
Proof.
  iIntros "HR". unfold generic_wp. simpl.
  iApply (wp_bind [AppRCtx _]). iApply (wp_generic_pack_acc [] [v]). simpl.
  iApply wp_app. simpl.
  iApply (wp_bind [LetCtx (Some "x") (Var "x")]).
  iApply wp_struct_load; [done|]. iIntros "!> _".
  iApply wp_let. simpl. iApply wp_value'. iFrame. done.
Qed.
```

`wp_generic_pack_acc` evaluates the internal argument tuple. `wp_app` enters the linked function; `wp_struct_load` extracts its parameter, and `wp_let` substitutes it. The final `iFrame` returns the untouched $R(v)$. This explains the uniformity: the body neither inspects the element nor requires any facts about its representation. The theorem is also independent of whether t is `Int64`, `Bool`, or a nested named application. Well-formed source types and native typing are separate checks from this semantic statement about values.

For example, one may take $R(w) = \ulcorner w = 42 \urcorner$, $R(w) = \ulcorner w = \mathsf{true} \urcorner$, or a predicate owning an object's fields. These are choices of logical predicates, not runtime type tests. The closed theorem `generic_main_adequate` applies this proof to the integer call returning 42, as described in Section 7.4.

9.3 Pairs: independent parameters and projection resources

`Pair<A, B>` is a struct with ordered fields `first: A` and `second: B`. Construction erases to a two-value tuple; no heap allocation is required. Its representation predicate is

$$\mathsf{PairRep}(R, S, p) \triangleq \exists a, b.\, \ulcorner p = \langle a, b \rangle \urcorner * R(a) * S(b).$$

The predicates R and S are independent, just like the type parameters A and B. The constructor theorem is `pair_construct_spec`:

$$R(a) * S(b) \vdash \mathsf{gwp}\,(\mathsf{Pair}\langle t_a, t_b \rangle (a, b))\,\{p.\, \mathsf{PairRep}(R, S, p)\}.$$

The proof only evaluates the fields and packages the two owned predicates:

```
Lemma pair_construct_spec (R S : val → iProp Σ) ta tb a b :
  R a -∗ S b -∗ generic_wp generic_layouts generic_runtime
    (GEConstruct (pair_ty ta tb) [GEVal a; GEVal b]) (pair_rep R S).
Proof.
  iIntros "Ha Hb". unfold generic_wp. simpl.
  iApply (wp_generic_pack_acc [] [a; b]). simpl. iExists a, b. iFrame. done.
Qed.
```

The source functions `firstOf<A, B>` and `secondOf<A, B>` project indexes zero and one of the explicitly named `Pair` layout. Their contracts are

$$\begin{aligned}
&\mathsf{PairRep}(R, S, p) \vdash \mathsf{gwp}\,(\mathsf{firstOf}\langle t_a, t_b \rangle (p)) \\
&\qquad\qquad \{w.\, \exists b.\, \ulcorner p = \langle w, b \rangle \urcorner * R(w) * S(b)\}, \\
&\mathsf{PairRep}(R, S, p) \vdash \mathsf{gwp}\,(\mathsf{secondOf}\langle t_a, t_b \rangle (p)) \\
&\qquad\qquad \{w.\, \exists a.\, \ulcorner p = \langle a, w \rangle \urcorner * R(a) * S(w)\}.
\end{aligned}$$

The equality identifies the returned component, and each component resource occurs once. In particular the first contract does not return $\mathsf{PairRep}(R, S, p) * R(w)$, which would duplicate an arbitrary resource. The checked proof of the first projection is

```
Lemma pair_first_spec (R S : val → iProp Σ) ta tb p :
  pair_rep R S p -∗ generic_wp generic_layouts generic_runtime
    (GECall "firstOf" [ta; tb] [GEVal p])
    (λ w, ∃ b, ⌜p = StructV [w; b]⌝ ∗ R w ∗ S b).
Proof.
  iIntros "Hp". iDestruct "Hp" as (a b ->) "[Ha Hb]".
```

```
  unfold generic_wp. simpl. iApply (wp_bind [AppRCtx _]).
  iApply (wp_generic_pack_acc [] [StructV [a; b]]). simpl. iApply wp_app. simpl.
  iApply (wp_bind [LetCtx (Some "p") (StructLoad (Var "p") 0)]).
  iApply wp_struct_load; [done|]. iIntros "!> _".
  iApply wp_let. simpl. iApply wp_struct_load; [done|].
  iIntros "!> _". iExists b. iFrame. done.
Qed.
```

The second proof uses the same argument at index one. Copying the struct value copies its stored values; it does not create another ownership permission for an object referenced by either value.

9.4 Boxes: allocation and reading

`Box<T>` is a final class with one immutable source field, `value: T`. Its runtime representation is an object reference. Writing $o \mapsto_o [0]v$ for `ObjId o` owning field zero, define

$$\mathsf{BoxRep}(R, p) \triangleq \exists o, v.\ \ulcorner p = \mathsf{obj}(o) \urcorner * o \mapsto_o [0]v * R(v).$$

This owns both the box's field and whatever resources describe its element. The field's source immutability does not make either ownership predicate persistent in this Iris model.

`box_construct_spec` and `box_get_spec` state

$$R(v) \vdash \mathsf{gwp}\,(\mathsf{Box}\langle t \rangle(v))\,\{p.\ \mathsf{BoxRep}(R, p)\},$$
$$\mathsf{BoxRep}(R, p) \vdash \mathsf{gwp}\,(p.\mathsf{get}())\ \{w.\ \exists o.\ \ulcorner p = \mathsf{obj}(o) \urcorner * o \mapsto_o [0]w * R(w)\}.$$

Construction uses `wp_new` to obtain the fresh field ownership and combines it with $R(v)$. Reading uses `wp_field_load`, which returns that same field ownership. Neither operation duplicates R. The implementation proofs are

```
Lemma box_construct_spec (R : val → iProp Σ) t v :
  R v -∗ generic_wp generic_layouts generic_runtime
    (GEConstruct (box_ty t) [GEVal v]) (box_rep R).
Proof.
  iIntros "HR". unfold generic_wp. simpl. iApply wp_generic_new_step. simpl.
  iApply wp_new. iIntros (o) "[Ho _]". iExists o, v. iFrame. done.
Qed.

Lemma box_get_spec (R : val → iProp Σ) t p :
  box_rep R p -∗ generic_wp generic_layouts generic_runtime
    (GEMethod (box_ty t) (GEVal p) "get" [] [])
    (λ w, ∃ o, ⌜p = LitV (LitObj o)⌝ ∗ ObjId o ↦o[0] w ∗ R w).
Proof.
  iIntros "Hp". iDestruct "Hp" as (o v ->) "[Ho HR]".
  unfold generic_wp. simpl. iApply (wp_bind [AppRCtx _]).
  iApply (wp_generic_pack_acc [] [LitV (LitObj o)]). simpl. iApply wp_app. simpl.
  iApply (wp_bind [LetCtx (Some "self") (FieldLoad (Var "self") 0)]).
  iApply wp_struct_load; [done|]. iIntros "!> _".
  iApply wp_let. simpl. iApply (wp_field_load with "Ho").
  iIntros "!> _ Ho". iExists o. iFrame. done.
Qed.
```

9.5 A type-changing method and its callback contract

In `Box<T>.map<U>`, `T` is fixed by the receiver and `U` is chosen at the call. The callback has type `(T) -> U`, and the result has type `Box<U>`. The method reads the old value, calls the callback once, and allocates a new box for its result. Its erased body, after parameter substitution, is

$$\mathsf{New}\ []\ [\mathsf{App}\ (\mathsf{Val}\ f)\ (\mathsf{FieldLoad}\ (\mathsf{obj}(o))\ 0)].$$

For input and output predicates R, S, the required callback contract is

$$\mathsf{Callback}(R, S, f)\ \triangleq\ \forall a.\ R(a) \mathrel{-\!\!*} \mathsf{wp}\,(f\ a)\,\{S\}.$$

It provides one invocation, consuming the input resource and supplying the output resource. It does not assert that the callback is pure or that its contract is duplicable. The additional Coq premise $\forall x, w.\ \mathsf{subst_val}\ x\ w\ f = f$ requires a closed callback value, so unpacking the method's parameters cannot capture free variables in it.

Under that premise, `box_map_spec` is

$$o \mapsto_o [0]v * R(v) * \mathsf{Callback}(R, S, f) \vdash \mathsf{gwp}\,(\mathsf{obj}(o).\mathsf{map}\langle u\rangle(f)) \\ \{p.\ o \mapsto_o [0]v * \mathsf{BoxRep}(S, p)\}.$$

The old box keeps the same field value. Its element resource $R(v)$ has gone to the callback; the result box receives S. Thus the postcondition deliberately retains the old field, rather than asserting a second $\mathsf{BoxRep}(R, \mathsf{obj}(o))$.

The complete checked statement and proof make the ownership flow explicit:

```
Lemma box_map_spec (R S : val → iProp Σ) t u o v f :
  (∀ x w, subst_val x w f = f) →
  ObjId o ↦ₒ[0] v -∗ R v -∗
  (∀ a, R a -∗ WP App (Val f) (Val a) {{ S }}) -∗
  generic_wp generic_layouts generic_runtime
    (GEMethod (box_ty t) (GEVal (LitV (LitObj o))) "map" [u] [GEVal f])
    (λ p, ObjId o ↦ₒ[0] v ∗ box_rep S p).
Proof.
  intros Hclosed. iIntros "Ho HR Hf". unfold generic_wp. simpl.
  iApply (wp_bind [AppRCtx _]). iApply (wp_generic_pack_acc [] [LitV (LitObj o); f]). simpl.
  iApply wp_app. simpl.
  repeat rewrite Hclosed.
  iApply (wp_bind [LetCtx (Some "self")
    (Let (Some "f") (StructLoad (Val (StructV [LitV (LitObj o); f])) 1)
      (New [] [App (Var "f") (FieldLoad (Var "self") 0)]))]).
  iApply wp_struct_load; [done|]. iIntros "!> _".
  iApply wp_let. simpl. repeat rewrite Hclosed.
  iApply (wp_bind [LetCtx (Some "f")
    (New [] [App (Var "f") (FieldLoad (Val (LitV (LitObj o))) 0)])]).
  iApply wp_struct_load; [done|]. iIntros "!> _".
  iApply wp_let. simpl.
  iApply (wp_bind [AppRCtx f; NewCtx [] []]).
  iApply (wp_field_load with "Ho"). iIntros "!> _ Ho".
  iApply (wp_bind [NewCtx [] []]).
  iApply (wp_wand with "[HR Hf]").
  { iApply ("Hf" with "HR"). }
  iIntros (w) "HS". iApply wp_generic_new_step. simpl.
  iApply wp_new. iIntros (o') "[Ho' _]". iFrame "Ho".
  iExists o', w. iFrame. done.
Qed.
```

After tuple unpacking, `wp_field_load` obtains the element while preserving `Ho`, the old field assertion. The subproof supplied with `[HR Hf]` gives only the element resource and callback contract to the callback. `Ho` is framed across it. Finally, `wp_new` creates `Ho'`, and the proof packages `Ho'` with `HS` into the result representation.

For a concrete ownership example, let $R = \mathsf{BoxRep}(P, -)$ describe an inner box and let the callback return that inner box unchanged, so $S = R$. Mapping an outer box produces a new outer object containing the same inner reference. The inner field ownership is transferred to the new outer representation. The old outer field is still owned, but one cannot claim that both outer boxes separately own the same inner box. If R is persistent, such as a pure integer-value assertion, the usual Iris persistence rules allow it to be retained. That is an extra property of the chosen predicate, not an assumption hidden in `map`.

9.6 Concrete and nested instances

The runtime suite uses both primitive and nested types. The following fragments illustrate the calls made possible by the declarations:

```
let n: Int64 = genericId<Int64>(42)
let b: Bool = genericId<Bool>(true)
let p = Pair<Int64, Bool>(8, true)
let first: Int64 = firstOf<Int64, Bool>(p)
let second: Bool = secondOf<Int64, Bool>(p)

let source = Box<Int64>(5)
let target: Box<Bool> = source.map<Bool>(
    { x: Int64 => 0 <= x })

let nested = genericId<Box<Pair<Int64, Bool>>>(
    Box<Pair<Int64, Bool>>(p))
let result: Box<Box<Bool>> = nested.map<Box<Bool>>(
    { x: Pair<Int64, Bool> => Box<Bool>(x.second) })
```

Here the method parameter is instantiated to `Bool` in the first map and to `Box<Bool>` in the second; it differs from the receiver's parameter in both cases.

One can express the nested representation by setting $R_8(v) = \ulcorner v = 8 \urcorner$, $R_{\mathsf{true}}(v) = \ulcorner v = \mathsf{true} \urcorner$, and $P(p) = \mathsf{PairRep}(R_8, R_{\mathsf{true}}, p)$. Then $\mathsf{BoxRep}(P, b)$ describes the nested input box. For the last callback, take $S(w) = \mathsf{BoxRep}(R_{\mathsf{true}}, w)$: projecting the second component and allocating its Boolean box supplies the callback obligation. The map rule returns the outer field of the old box and $\mathsf{BoxRep}(S, \mathit{result})$. This is how the uniform contracts can be composed; the particular native test is a runtime check, not a separately named closed adequacy theorem for this whole fragment.

`repeatIdentity<T>` also exercises direct generic recursion. Its native tests instantiate it at integers, booleans, and pairs. The suite checks recursion and printing; this example does not supply a separate general termination proof for that function.

9.7 Generated source and extraction checks

The emitted program below is read directly from the generated file. The small identity-lambda wrappers in field accesses force the native compiler to check the annotated receiver type. Similar wrappers check lambda result annotations. They are not used to select layouts by value: the source explicitly names the declaration and field index.

```
// Generated from checked CJR generic source; do not edit.
import std.core.Array as CjrBuffer
public struct Pair<A, B> {
public let first: A
public let second: B
public init(first: A, second: B) {
this.first = first
this.second = second
}
}
public class Box<T> {
public let value: T
public init(value: T) {
this.value = value
}
public func get(): T {
let self = this
return (({ cjrTypedValue: Box<T> => cjrTypedValue })(self)).value
}
public func map<U>(f: (T) -> U): Box<U> {
let self = this
return Box<U>((f)((({ cjrTypedValue: Box<T> => cjrTypedValue })(self)).value))
}
}
public struct OtherPair<A, B> {
public let left: A
public let right: B
public init(left: A, right: B) {
this.left = left
this.right = right
}
}
public class OtherBox<T> {
public let item: T
public init(item: T) {
this.item = item
}
}
public func genericId<T>(x: T): T {
return x
}
public func genericIdAgain<T>(x: T): T {
return x
}
public func firstOf<A, B>(p: Pair<A, B>): A {
return (({ cjrTypedValue: Pair<A, B> => cjrTypedValue })(p)).first
}
public func secondOf<A, B>(p: Pair<A, B>): B {
return (({ cjrTypedValue: Pair<A, B> => cjrTypedValue })(p)).second
}
public func repeatIdentity<T>(x: T, n: Int64): T {
return if ((n  <=  0)) { x } else { repeatIdentity<T>(x, (n  -  1)) }
}
```

```
public func makePair<A, B>(a: A, b: B): Pair<A, B> {
return Pair<A, B>(a, b)
}
public func makeBox<T>(x: T): Box<T> {
return Box<T>(x)
}
public func positiveBox(n: Int64): Bool {
return ({ =>
let b = Box<Int64>(n)
(({ cjrTypedValue: Box<Bool> => cjrTypedValue })((({ cjrTypedValue: Box<Int64> =>
    cjrTypedValue })(b)).map<Bool>({ x: Int64 => ({ cjrTypedValue: Bool => cjrTypedValue
     })((0  <=  x)) }))).get()
})()
}
public func strictAnd(f: (Unit) -> Bool): Bool {
return ({ cjrLeft: Bool, cjrRight: Bool => cjrLeft && cjrRight })(false, (f)(()))
}
main() {
genericId<Int64>(42)
0
}
```

The checked front end returns `Extracted` only after structural validation; otherwise `Rejected` contains diagnostics. It checks declaration names, type-variable scopes, type and value arities, named receiver layouts, field indexes, and supported forms. For example, the regression suite includes these failures:

```
genericId with no type argument:
  wrong type arity: genericId
missing<Int64>(1), with no such declaration:
  unresolved function (declare dependencies first): missing
Box<Bool>() with no field argument:
  wrong constructor arity: Box
Box<T>.map<T> rebinding its owner's parameter:
  invalid, shadowed, or duplicate type parameters
```

This listing shows diagnostics rather than runnable Cangjie source. Checking type compatibility of ordinary arguments and results is left to Cangjie compilation. Tests deliberately compile invalid receiver and lambda annotations and require rejection. Distinct declarations with identical bodies or equal-sized layouts have separate explicit references; the printer does not choose between them by inspecting those bodies or sizes.

Evaluation order is part of this boundary. Calls and constructors keep left-to-right argument order; the receiver precedes method arguments. CJR's Boolean conjunction evaluates both operands, so the generic printer uses a helper call to preserve that evaluation before applying native `&&`. The `strictAnd` runtime case checks the right-hand callback's effect even when the left-hand value is false.

9.8 Validation and the scope of the guarantees

`example/generics/rocq/generic_tests.v` checks simultaneous and nested type substitution, type-parameter scopes, instantiation erasure, local substitution under binders, and extraction rejection cases in the Rocq kernel. `example/generics/test/generics_test.cj` supplies ten native test

cases, including heterogeneous pairs, type-changing maps, nested instances, identical-body declarations, and direct recursion. Five compiler-rejection tests in `tests/test_generic_extraction.py` check unrelated receiver layouts, incorrect receiver type arguments, and incorrect lambda result types, buffer receiver annotations, and buffer write types against the Cangjie 1.0.5 toolchain. The polymorphic-array chapter of the CJR Collections tutorial extends the examples with managed polymorphic arrays and a paired benchmark.

`make verify` includes the erasure laws, generic logic, representation contracts, and generic adequacy results in the assumption audit. `make test` includes the native generic and existing runtime suites. Existing monomorphic examples retain their original printer and proofs; the generic addition does not migrate their data structures.

The result is uniform Iris specifications for the supplied implementations and safe execution of erased programs under their proved preconditions. There is no general typing-soundness or relational-parametricity theorem. The printer and native compiler remain in the translation trust boundary, and the unbounded-integer model still requires care at `Int64` boundaries. Interface bounds, inheritance, higher-rank polymorphism, and generic raw-memory allocation are outside the implemented source fragment.

10 Limitations and target-code risks

CJR integers are unbounded mathematical integers, while generated Int64 values have machine bounds. Callers must satisfy the stated arithmetic and memory bounds. The logic specifies a sequential fragment and does not prove a full Cangjie type-soundness theorem, native compiler correctness, or the behavior of operating-system and frontend code. Generic specifications quantify over resource representations; they are not a universal relational-parametricity result. Checked kernels and conditional refinement contracts must be read with their hypotheses and documented coverage. Proofs were not rerun during this migration, and runtime tests supply separate empirical evidence.

11 Migration and current runtime evidence

Proofs were not revalidated during this repository split. Historical proof reports retain their original dates and provenance. Runtime timings measure generated native programs and do not establish cryptographic security or compiler correctness.

11.1 Measurement protocol

Timings are from optimized Cangjie 1.0.5 builds on the machine recorded in each metadata file. Each suite uses its checked-in configuration, warmups, independent processes, and checksum validation. Medians and interquartile ranges describe repeated samples; they are not universal guarantees.

11.2 benchmark/results

Recorded 2026-10-29T14:13:10.640557+00:00. 8 workload groups. Values below are ns per harness-defined operation. Full results, provenance, and charts are in the local HTML report.

Workload / implementation	Median	IQR

arithmetic / 4096 / CJR	0.45	0.00
arithmetic / 4096 / Handwritten	0.45	0.00
block_sum / 4096 / CJR	0.69	0.02
block_sum / 4096 / Handwritten	0.34	0.00
generic_callback / 4096 / CJR	0.34	0.00
generic_callback / 4096 / Handwritten	0.34	0.00
swap / 4096 / CJR	0.45	0.00
swap / 4096 / Handwritten	0.45	0.00

References

[1] Cangjie documentation, version 1.0.0. `https://docs.cangjie-lang.cn/en/docs/1.0.0/`.

[2] John C. Reynolds. Separation logic: A logic for shared mutable data structures. In *Proceedings of LICS*, pages 55–74. IEEE, 2002.

[3] Peter W. O'Hearn, John C. Reynolds, and Hongseok Yang. Local reasoning about programs that alter data structures. In *Proceedings of CSL*, volume 2142 of *LNCS*, pages 1–19. Springer, 2001.

[4] Ralf Jung, David Swasey, Filip Sieczkowski, Kasper Svendsen, Aaron Turon, Lars Birkedal, and Derek Dreyer. Iris: Monoids and invariants as an orthogonal basis for concurrent reasoning. In *Proceedings of POPL*, pages 637–650. ACM, 2015.

[5] Robbert Krebbers, Ralf Jung, Aleš Bizjak, Jacques-Henri Jourdan, Derek Dreyer, and Lars Birkedal. The essence of higher-order concurrent separation logic. In *Proceedings of ESOP*, volume 10201 of *LNCS*, pages 696–723. Springer, 2017.

[6] Ralf Jung, Robbert Krebbers, Jacques-Henri Jourdan, Aleš Bizjak, Lars Birkedal, and Derek Dreyer. Iris from the ground up: A modular foundation for higher-order concurrent separation logic. *Journal of Functional Programming*, 28:e20, 2018.

[7] Robbert Krebbers, Amin Timany, and Lars Birkedal. Interactive proofs in higher-order concurrent separation logic. In *Proceedings of POPL*, pages 205–217. ACM, 2017.

[8] Lars Birkedal and Aleš Bizjak. Lecture notes on Iris: Higher-order concurrent separation logic. `https://iris-project.org/pdfs/iris-lecture-notes.pdf`.

[9] The Iris Project. The Iris Coq development, version 4.4.0, including HeapLang. `https://gitlab.mpi-sws.org/iris/iris`.

[10] The Coq Development Team. The Coq proof assistant, version 8.20.1. `https://coq.inria.fr/`.

[11] The Iris Project. coq-std++, version 1.12.0. `https://gitlab.mpi-sws.org/iris/stdpp`.

[12] Cangjie development guide, version 1.0.5. Throwing and handling exceptions (Chinese-language edition). `https://docs.cangjie-lang.cn/docs/1.0.5/dev-guide/source_zh_cn/error_handle/handle.html`.

[13] Paulo Emílio de Vilhena and François Pottier. A separation logic for effect handlers. *Proceedings of the ACM on Programming Languages*, 5(POPL), article 33, pages 1–28, 2021. `https://doi.org/10.1145/3434314`.

[14] Paulo Emílio de Vilhena and François Pottier. A type system for effect handlers and dynamic labels. In *Proceedings of ESOP*, volume 13990 of *LNCS*, pages 225–252. Springer, 2023. https://doi.org/10.1007/978-3-031-30044-8_9.

[15] Dan Frumin, Robbert Krebbers, and Lars Birkedal. ReLoC Reloaded: A mechanized relational logic for fine-grained concurrency and logical atomicity. *Logical Methods in Computer Science*, 17(3), article 9, pages 1–59, 2021. https://doi.org/10.46298/LMCS-17(3:9)2021.

[16] Glen Mével, Jacques-Henri Jourdan, and François Pottier. Cosmo: A concurrent separation logic for Multicore OCaml. *Proceedings of the ACM on Programming Languages*, 4(ICFP), article 96, pages 1–29, 2020. https://doi.org/10.1145/3408978.

[17] Jonas Kastberg Hinrichsen, Jesper Bengtson, and Robbert Krebbers. Actris 2.0: Asynchronous session-type based reasoning in separation logic. *Logical Methods in Computer Science*, 18(2), article 16, pages 1–64, 2022. https://doi.org/10.46298/LMCS-18(2:16)2022.

[18] Paulo Emílio de Vilhena, Simcha van Collem, Ines Wright, and Robbert Krebbers. A relational separation logic for effect handlers. *Proceedings of the ACM on Programming Languages*, 10(POPL), article 34, pages 981–1009, 2026. https://doi.org/10.1145/3776676. Rocq formalization: https://github.com/DeVilhena-Paulo/blaze.